Commun. Comput. Phys.
doi: 10.4208/cicp.OA-2025-0228

Vol. **x**, No. x, pp. 1-34
xx 2026

Neural Network Based Dynamic Domain Decomposition for Multiscale Kinetic Equations

Qiming Wang[1], Yi Cai[1] and Tao Xiong[2,*]

[1] *School of Mathematical Sciences, Xiamen University, Xiamen, Fujian 361005, P.R. China.*
[2] *School of Mathematical Sciences, University of Science and Technology of China, Hefei, Anhui 230026, P.R. China.*

Received 02 Sep 2025; Accepted (in revised version) 16 Apr 2026

Abstract. Complex gas flows with a multi-scale nature pose significant challenges. A hierarchical scheme using a hybrid kinetic/fluid solver can effectively address this challenge. One such approach is a dynamic domain decomposition method based on moment realizability matrices. Despite its potential, this method faces complexities in tuning manual parameters and calculating moment realizability matrices. In this study, we propose a Fully Connected Neural Network (FCNN)-based classifier which numerically performs better than traditional moment realizability matrix methods. This classifier distinguishes between near-equilibrium and non-equilibrium flow states using macroscopic quantities derived from numerical solutions of the BGK equations. By training on short-term numerical solutions, the classifier effectively predicts long-term evolution in the BGK equations, eliminating the need for calculating macroscopic derivatives and manually tuning parameters. Numerical experiments have confirmed the good performance of the FCNN-based classifier.

AMS subject classifications: 76P05, 68T07, 62H30, 65M08

Key words: neural network, kinetic equation, domain decomposition, multi-scale, hybrid scheme.

1 Introduction

Many physical problems, such as hypersonic aerodynamic systems, involve boundary layers or transitional regimes that cannot be solved using standard fluid models. Therefore, a kinetic model is required to precisely describe the complex phenomena around the boundary. However, simulating a kinetic model is computationally expensive, making its use feasible only in localized regions. An effective approach is to design hybrid kinetic/fluid schemes with domain decomposition that can automatically and accurately

*Corresponding author. *Email addresses:* qimingwang@stu.xmu.edu.cn (Q. Wang), yicaim@stu.xmu.edu.cn (Y. Cai), taoxiong@ustc.edu.cn (T. Xiong)

identify the kinetic and fluid zones. In this case, we only need to solve the kinetic model around the kinetic boundary layers or shocks, while benefiting from the low computational cost of fluid schemes elsewhere.

Several works on hybrid methods directly based on the local Knudsen number for multi-scale kinetic equations already exist in the literature [1–3]. Another hydrodynamic breakdown indicator based on the viscous and heat flux of the Navier–Stokes equations was introduced by S. Tiwari [4] and has been used with various deterministic kinetic and hydrodynamic solvers [5–9]. Recently, F. Filbet and T. Rey [10] proposed a domain decomposition indicator based on moment realizability matrices, a concept first introduced by D. Levermore, W. Morokoff, and B. Nadiga [11]. They showed the need for criteria to transition between hydrodynamic and kinetic regimes and explored finite volume schemes for both the Boltzmann equation and hydrodynamic equations. This criterion has recently been used by T. Xiong and J.-M. Qiu [12] to form high-order discontinuous Galerkin schemes for the BGK equation under a micro-macro decomposition framework. This work was later extended to the two-dimensional case by F. Filbet and T. Xiong [13]. However, this criterion requires significant computational effort since it is based on comparing local eigenvalues in each cell, making the cost non-negligible in high dimensions. Additionally, the values of the manual parameters need constant tuning for different problems.

The aim of this paper is to train a neural network for a dynamic domain decomposition, where the training labels are obtained from the domain decomposition method based on moment realizability matrices. One of the foundational works in neural network research is Frank Rosenblatt's work on perceptrons [14] in 1958, which opened up a wide range of research and applications in classification and pattern recognition. Fully Connected Neural Networks (FCNNs), which is also known as dense neural networks, are widely utilized in various fields for classification tasks due to their versatility and effectiveness. In finance, FCNNs are used for credit scoring and fraud detection [15, 16]. In healthcare, FCNNs are superior for disease prediction and medical diagnosis [17, 18]. Marketing also benefits from FCNNs for customer segmentation and market basket analysis [19]. Besides, Convolutional Neural Networks (CNNs) specialize in image and video processing tasks [20–24], while Recurrent Neural Networks (RNNs) are designed for sequential data [25, 26].

In our classification problem, which falls under supervised learning, we consider to use FCNNs instead of CNNs or RNNs. G. Cybenko [27] proved that any continuous function can be uniformly approximated by a continuous neural network with a single hidden layer and an arbitrary continuous sigmoidal nonlinearity. Additionally, neural networks are adept at handling complex non-linear relationships, making them suitable for a wide array of supervised learning tasks [28]. CNNs compress the high-dimensional representation of raw data into lower-dimensional feature representations through convolutional and pooling layers. Since our network input is not large, dimensionality reduction is unnecessary. Furthermore, our network output depends on local spatial information and is unrelated to time sequences. Therefore, FCNNs are a more appropriate choice due

to their simplicity and effectiveness for our specific feature set and problem structure. Moreover, J. Yu and J. Hesthaven proposed a data-driven artificial viscosity model [29] for shock capturing in discontinuous Galerkin methods. They established a supervised learning model which takes numerical solutions around each target spatial element as inputs and produces a smoothness indicator as output. This work inspired us to construct an FCNN, with the solution data of the Boltzmann equation as inputs and the domain indicator of the corresponding unit as output.

We use FCNNs as surrogate classifiers for the most probable flow regime in the training data. FCNNs accept macroscopic quantities such as density, mean velocity, temperature, and pressure as inputs and return domain indicators as labels. By collecting solutions and their corresponding labels from [12] and [13] in a short period, we aim for the FCNN to learn the majority of features and predict the values of domain indicators effectively even after the long-term evolution of the BGK equation. Recently, Xiao et al. proposed a neural network classifier [30] to detect near-equilibrium and non-equilibrium flow regimes based on local flow conditions. Their approach formulates regime detection as a binary classification problem and derives labels from the deviation between the particle distribution function and its Chapman–Enskog-based hydrodynamic approximation. This design enables a physics-consistent criterion for continuum breakdown.

From a methodological perspective, the approach of [30] differs from the present work in both input representation and label construction. Their neural network takes inputs as macroscopic moments together with their spatial derivatives and collision-related quantities, aiming to implicitly capture non-equilibrium information at the distribution-function level. Correspondingly, the labels are defined through distribution-level deviations relative to Chapman–Enskog expansions

$$\mathcal{R}=\begin{cases}1, & d>\epsilon\\ 0, & d\leq\epsilon\end{cases}, \quad d=\frac{||f_{\mathrm{NS}}-f_{\mathrm{ref}}||_2}{\rho}, \tag{1.1}$$

where f_{ref} is a given distribution function and ϵ is an acceptable error value that needs to be defined manually. In contrast, the proposed moment realizability-matrix-based method is designed as a purely macroscopic surrogate for domain decomposition. The network inputs consist only of standard macroscopic quantities, while the training labels are constructed from a moment realizability matrix that reflects the admissibility of fluid closures. This strategy avoids the explicit use of distribution functions or gradient-based features and provides a direct mapping from macroscopic states to domain indicators.

We will perform several numerical tests on physically relevant problems with shocks or boundary layers in Section 4, such as the 1D Sod problem, 1D blast wave problem, 1D Lax problem, 2D Riemann problem, and 2D ghost effect. The results will demonstrate the efficiency and effectiveness of our proposed approach. As a result, our method emphasizes simplicity and efficiency at the macroscopic level, while retaining consistency with kinetic–fluid coupling principles. These differences place the proposed method more robustly for a dynamic domain decomposition in such a machine-learning-assisted multiscale solvers.

The rest of the paper is organized as follows. In Section 2, we introduce the form of the BGK equation and recall the hydrodynamic limits of the multi-scale kinetic equation based on Chapman–Enskog expansions. In Section 3, we describe the domain decomposition method based on moment realizability matrices and introduce the method based on FCNN. Numerical tests are presented in Section 4. Conclusions and future work are discussed in Section 5.

2 Formulation and moment realizability matrix-based domain decomposition

In this section, we first briefly describe the multi-scale kinetic equation, specifically the BGK equation, and then we give a brief introduction of the domain decomposition method based on moment realizability matrices.

2.1 Formulation of the BGK equation

In this work, we start with the simple BGK equation [31]:

$$\begin{cases} \partial_t f + \mathbf{v}\cdot\nabla_{\mathbf{x}} f = \dfrac{\nu}{\varepsilon}(M_U - f) \\ f(0,\mathbf{x},\mathbf{v}) = f_0(\mathbf{x},\mathbf{v}), \end{cases} \tag{2.1}$$

where $f = f(t,\mathbf{x},\mathbf{v})$ is the distribution function of particles that depends on time $t>0$, position $\mathbf{x}\in\Omega\subset\mathbb{R}^{d_x}$ and velocity $\mathbf{v}\in\mathbb{R}^3$. The open set Ω is a bounded Lipschitz-continuous domain in $\mathbb{R}^{d_x}$, which satisfys some boundary conditions. f_0 is a non-negative initial function. ν is a collision frequency. The parameter $\varepsilon>0$ is the dimensionless Knudsen number, which represents the ratio of the mean free path of particles over a typical length scale such as the spatial domain. M_U is the local Maxwellian defined by

$$M_U = M_U(t,\mathbf{x},\mathbf{v}) = \frac{\rho(t,\mathbf{x})}{(2\pi T(t,\mathbf{x}))^{3/2}} \exp\left(-\frac{|\mathbf{v}-\mathbf{u}(t,\mathbf{x})|^2}{2T(t,\mathbf{x})}\right). \tag{2.2}$$

The density ρ, mean velocity u and temperature T are macroscopic moments of the distribution function f, which can be computed as

$$\rho = \int_{\mathbb{R}^3} f(\mathbf{v})\,d\mathbf{v}, \quad \mathbf{u} = \frac{1}{\rho}\int_{\mathbb{R}^3} \mathbf{v} f(\mathbf{v})\,d\mathbf{v}, \quad T = \frac{1}{3\rho}\int_{\mathbb{R}^3} |\mathbf{v}-\mathbf{u}|^2 f(\mathbf{v})\,d\mathbf{v}. \tag{2.3}$$

For the collision frequency, it can be taken as $\nu = \frac{2}{\sqrt{\pi}}\rho$ [32].

We denote $m = m(\mathbf{v}) := \left(1,\mathbf{v},\frac{1}{2}|\mathbf{v}|^2\right)^\top$ and $\langle g\rangle := \int_{\mathbb{R}^3} g(\mathbf{v})\,d\mathbf{v}$. Let U denote the conservative macroscopic components of density, momentum and energy, which correspond to the first three moments of the distribution function f

$$U := (\rho,\rho\mathbf{u},E)^\top = \int_{\mathbb{R}^3} m(\mathbf{v}) f(\mathbf{v})\,d\mathbf{v}, \tag{2.4}$$

where $E=\frac{1}{2}\rho|\mathbf{u}|^2+\frac{3}{2}\rho T$. The BGK operator (M_U-f) satisfies the conservation [31] of mass, momentum and energy, since

$$\partial_t U+\nabla_{\mathbf{x}}\langle \mathbf{v}mf\rangle=\mathbf{0}. \tag{2.5}$$

Moreover, it satisfies the entropy dissipation [31]

$$\langle (M_U-f)\log f\rangle \leq 0. \tag{2.6}$$

2.2 Moment realizability matrix-based decomposition method

In local thermodynamic equilibrium, the system is well described by a finite set of macroscopic quantities, and the moment realizability matrix approaches the identity. When the system departs from equilibrium, as in shock layers or rarefied regimes, this matrix no longer approximates the identity and its eigenvalues deviate significantly from unity.

To determine the appropriate regions for applying kinetic versus hydrodynamic models, F. Filbet and T. Xiong proposed a physically motivated criterion based on the moment realizability matrix [13]. This approach evaluates the local structure of the distribution function by considering the macroscopic quantities, including the density, the mean velocity and the temperature, along with their spatial gradients. From fluid to kinetic, they derived a simplified indicator that captures the deviation between the first order compressible Navier–Stokes equations and the Burnett equations. This indicator is expressed as

$$\lambda_{\varepsilon^2}(t,\mathbf{x}):=\varepsilon^2\left(\frac{|\nabla_{\mathbf{x}}T|^2}{T}+|\nabla_{\mathbf{x}}\mathbf{u}|^2+\sqrt{\left(|\Delta_{\mathbf{x}}\mathbf{u}|^2+|\Delta_{\mathbf{x}}\rho/\rho|^2\right)(1+T^2)}\right), \tag{2.7}$$

where ε is the Knudsen number.

The selection between kinetic and hydrodynamic models at a specific point $(t,\mathbf{x})$ is based on two distinct criteria, depending on the direction of the model transition.

From fluid to kinetic, the compressible Navier–Stokes model is considered invalid at a spatial location $(t,\mathbf{x})$ if the following condition is satisfied:

$$|\lambda_{\varepsilon^2}(t,\mathbf{x})|>\eta_0, \tag{2.8}$$

where $\lambda_{\varepsilon^2}(t,\mathbf{x})$ is the indicator evaluated at the cell center, and η_0 is a manually prescribed threshold.

From kinetic to fluid, if a kinetic representation at $(t,\mathbf{x})$ sufficiently approximates the hydrodynamic behavior, the model can transition back to a fluid description. This is determined by the L^2-distance between the distribution function f and the first-order Chapman–Enskog expansion:

$$\|f(t,\mathbf{x},\cdot)-f_2(t,\mathbf{x},\cdot)\|_{L^2}\leq\delta_0, \tag{2.9}$$

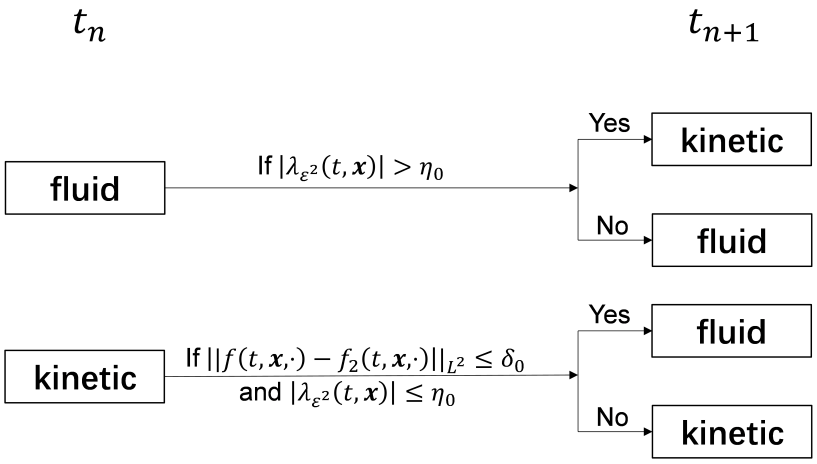

Figure 1: Criteria of domain decomposition method based on moment realizability matrix.

where $f_2 = M[1+\varepsilon g^{(1)}]$ denotes the expansion truncated at first order, with

$$g^{(1)} = -A\frac{\partial_x T}{\sqrt{T}}M, \quad A = \left(\frac{|v-u|^2}{2T} - \frac{3}{2}\right)\frac{v-u}{\sqrt{T}}, \tag{2.10}$$

and δ_0 is a small parameter.

The overall switching strategy is illustrated in Fig. 1. For 1D cases, the strategy is similar in [12].

2.3 Calculation of macroscopic quantities and their derivatives

We focus on the calculation of the values of macroscopic quantities and their corresponding derivatives such as $\rho, \mathbf{u}, T$ and $\Delta_{\mathbf{x}}\rho, \nabla_{\mathbf{x}}\mathbf{u}, \Delta_{\mathbf{x}}\mathbf{u}, \nabla_{\mathbf{x}}T$. In [13], a three-point Lagrange interpolation has been used on the uniformed mesh.

We set a partition to be a rectangular $\Omega_x = [a,b]\times[c,d]$, $a = x_{\frac{1}{2}} < x_{\frac{3}{2}} < \cdots < x_{N_x+\frac{1}{2}} = b, c = y_{\frac{1}{2}} < y_{\frac{3}{2}} < \cdots < y_{N_y+\frac{1}{2}} = d$. Let $I_{i,j} = [x_{i-\frac{1}{2}}, x_{i+\frac{1}{2}}]\times[y_{i-\frac{1}{2}}, y_{i+\frac{1}{2}}]$ denotes an element with its length $\Delta x_i = x_{i+\frac{1}{2}} - x_{i-\frac{1}{2}}$ and $\Delta y_j = y_{j+\frac{1}{2}} - y_{j-\frac{1}{2}}$. We take the 1st Gauss point in each cell and use finite difference to compute the values.

We establish a three-point Lagrange interpolation formula $P(x)$ for $F(x,y)$ centered on (x_i, y_i) along $x-$direction. The interpolation along $y-$direction can be similarly defined.

$$\begin{aligned} P(x) =& F(x_i,y_i)\frac{(x-x_{i-1})(x-x_{i+1})}{(x_i-x_{i-1})(x_i-x_{i+1})} + F(x_{i-1},y_i)\frac{(x-x_i)(x-x_{i+1})}{(x_{i-1}-x_i)(x_{i-1}-x_{i+1})} \\ &+ F(x_{i+1},y_i)\frac{(x-x_i)(x-x_{i-1})}{(x_{i+1}-x_i)(x_{i+1}-x_{i-1})}, \end{aligned} \tag{2.11}$$

We also have the first order and second order derivative

$$P'(x)=\frac{f(x_i)(2x-x_{i-1}-x_{i+1})}{(x_i-x_{i-1})(x_i-x_{i+1})}+\frac{f(x_{i-1})(2x-x_i-x_{i+1})}{(x_{i-1}-x_i)(x_{i-1}-x_{i+1})}+\frac{f(x_{i+1})(2x-x_{i-1}-x_i)}{(x_{i+1}-x_{i-1})(x_{i+1}-x_i)}, \quad (2.12)$$

$$P''(x)=\frac{2f(x_i)}{(x_i-x_{i-1})(x_i-x_{i+1})}+\frac{2f(x_{i-1})}{(x_{i-1}-x_i)(x_{i-1}-x_{i+1})}+\frac{2f(x_{i+1})}{(x_{i+1}-x_{i-1})(x_{i+1}-x_i)}. \quad (2.13)$$

The form can be simplified on the uniformed mesh. We set $h=\Delta x=\Delta y$ as the size of the mesh along $x-$direction and $y-$direction, respectively. The values of macroscopic quantities $\rho,\mathbf{u},T$ can be computed by kinetic solver and hydrodynamic solver in [13]. Hence we give the formula of their derivatives

$$\begin{cases} |\Delta_{\mathbf{x}}\rho_{i,j}|^2=\dfrac{\left(\rho_{i-1,j}+\rho_{i+1,j}-2\rho_{i,j}\right)^2}{h^4}+\dfrac{\left(\rho_{i,j-1}+\rho_{i,j+1}-2\rho_{i,j}\right)^2}{h^4}, \\ |\nabla_{\mathbf{x}}\mathbf{u}_{i,j}|^2=\dfrac{||\mathbf{u}_{i+1,j}-\mathbf{u}_{i-1,j}||_2^2}{4h^2}+\dfrac{||\mathbf{u}_{i,j+1}-\mathbf{u}_{i,j-1}||_2^2}{4h^2}, \\ |\Delta_{\mathbf{x}}\mathbf{u}_{i,j}|^2=\dfrac{||\mathbf{u}_{i-1,j}+\mathbf{u}_{i+1,j}-2\mathbf{u}_{i,j}||_2^2}{h^4}+\dfrac{||\mathbf{u}_{i,j-1}+\mathbf{u}_{i,j+1}-2\mathbf{u}_{i,j}||_2^2}{h^4}, \\ |\nabla_{\mathbf{x}}T_{i,j}|^2=\dfrac{\left(T_{i+1,j}-T_{i-1,j}\right)^2}{4h^2}+\dfrac{\left(T_{i,j+1}-T_{i,j-1}\right)^2}{4h^2}. \end{cases} \quad (2.14)$$

We replace the value on the 1st Gauss point with the value on the grid point.

Remark 2.1. For 1D case [12], we increase the 0-th Chapman-Enskog expansion. Then we need to group all the computational cells into three classes: (1) Euler regime, (2) Navier-Stokes regime, (3) kinetic regime. We can find the detailed criterion in [12] which also has the identities above.

3 Neural network-based domain decomposition

We propose a data-driven approach using neural networks to automatically learn domain decomposition criterion from macroscopic quantities. This enables efficient and robust region classification between kinetic and fluid regimes without manual tuning. In this section, we will present the design and implementation of this neural classifier.

3.1 Training data preparation

From the last section, we already know the moment realizability matrix-based domain decomposition needs the values of macroscopic quantities and their derivatives. For 2D cases, we can compute the derivatives by (2.14), with cells in Fig. 2a. Thus, we intend to set the macroscopic quantities of the cells in Fig. 2a as the inputs to the network. In order to improve the performance of our neural network, we extracted additional information from four cells in the corners as input data. The cells we choose can be seen in Fig. 2b.

We extracted ρ, u_x, u_y, T, p on each cell, which represent the macroscopic density, the mean velocity in x-direction, the mean velocity in y-direction, the temperature and the pressure. We choose the pressure p since $p=\rho T$, which is also an important quantity. Also, we add ε and Δx, which represent the Knudsen number and the mesh size.

For each cell index (r,s), we define the corresponding vector composed of macroscopic quantities as

$$\mathbf{k}_2^{(r,s)} = \left[\rho^{(r,s)}, u_x^{(r,s)}, u_y^{(r,s)}, T^{(r,s)}, p^{(r,s)}\right]^{\mathsf{T}} \in \mathbb{R}^5. \tag{3.1}$$

The complete input vector $\mathbf{X}_{i,j}$ for the FCNN is then formed by concatenating the nine vectors, followed by ε and Δx:

$$\begin{aligned}\mathbf{X}_{i,j} = \Big(&\mathbf{k}_2^{(i-1,j+1)}, \mathbf{k}_2^{(i,j+1)}, \mathbf{k}_2^{(i+1,j+1)}, \mathbf{k}_2^{(i+1,j)}, \mathbf{k}_2^{(i,j)}, \mathbf{k}_2^{(i-1,j)},\\ &\mathbf{k}_2^{(i-1,j-1)}, \mathbf{k}_2^{(i,j-1)}, \mathbf{k}_2^{(i+1,j-1)}, \varepsilon, \Delta x\Big)^{\mathsf{T}} \in \mathbb{R}^{47}.\end{aligned} \tag{3.2}$$

We also construct a reversed-order input $\mathbf{X}_{i,j}^{\text{rev}}$, in which the scanning order of the nine cells is exactly inverted

$$\begin{aligned}\mathbf{X}_{i,j}^{\text{rev}} = \Big(&\mathbf{k}_2^{(i+1,j-1)}, \mathbf{k}_2^{(i,j-1)}, \mathbf{k}_2^{(i-1,j-1)}, \mathbf{k}_2^{(i-1,j)}, \mathbf{k}_2^{(i,j)}, \mathbf{k}_2^{(i+1,j)},\\ &\mathbf{k}_2^{(i+1,j+1)}, \mathbf{k}_2^{(i,j+1)}, \mathbf{k}_2^{(i-1,j+1)}, \varepsilon, \Delta x\Big)^{\mathsf{T}}.\end{aligned} \tag{3.3}$$

Note that the center cell $\mathbf{s}^{(i,j)}$ remains at the fifth position in both orderings, while the neighboring cells appear in reversed sequence. The label corresponding to each row of data is set to

$$Y_{i,j} = \begin{cases} 0, & \text{if } I_{i,j} \text{ is in the kinetic region,} \\ 1, & \text{if } I_{i,j} \text{ is in the Navier-Stokes region.} \end{cases} \tag{3.4}$$

	$I_{i,j+1}$	
$I_{i-1,j}$	$I_{i,j}$	$I_{i+1,j}$
	$I_{i,j-1}$	

(a) 2D original cells.

$I_{i-1,j+1}$	$I_{i,j+1}$	$I_{i+1,j+1}$
$I_{i-1,j}$	$I_{i,j}$	$I_{i+1,j}$
$I_{i-1,j-1}$	$I_{i,j-1}$	$I_{i+1,j-1}$

(b) 2D FCNN cells.

Figure 2: Requirement of 2D Cells for original/FCNN method. From left to right: (a) 2D original cells, (b) 2D FCNN cells. **Red color denotes the central cell, blue color denotes its neighboring cells.**

Algorithm 1 2D Indicator Computation Module.

Require: Computational cell $I_{i,j}$, macroscopic variables $\rho,\mathbf{u},T$, distribution function f, domain indicator $\text{ind}(t^{n-1},\mathbf{x})$ for the last time layer t^{n-1}
Ensure: Indicator value: 0 (kinetic region), 1 (Navier-Stokes region)
Initialize $\text{ind}(t^0,\mathbf{x}) \leftarrow 0$ ▷ Default: use kinetic solver
Step 1: Check hydrodynamic breakdown (fluid → kinetic)
if $\text{ind}(t^{n-1},\mathbf{x}) = 1$ **then**
  Compute spatial gradients and Laplacians $\Delta_x\rho,\nabla_x\mathbf{u},\Delta_x\mathbf{u},\nabla_x T$ by Eq. (2.14)
  Compute fluid breakdown indicator $\lambda_{\varepsilon^2}(t,\mathbf{x})$ by Eq. (2.7)
  if $|\lambda_{\varepsilon^2}(t,\mathbf{x})| > \eta_0$ **then** ▷ e.g., $\eta_0 = 10^{-3}$
    $\text{ind}(t^n,\mathbf{x}) \leftarrow 0$ ▷ Switch to kinetic solver
  end if
end if
Step 2: Check kinetic-to-fluid suitability (kinetic → fluid)
if $\text{ind}(t^{n-1},\mathbf{x}) = 0$ **then**
  Compute first-order Chapman-Enskog expansion: $f_2 \leftarrow \mathcal{M}[1+\varepsilon g^{(1)}]$ by Eq. (2.10)
  Compute error: $\text{err} \leftarrow \|f - f_2\|_{L^2}$
  if $\text{err} \le \delta_0$ **then** ▷ e.g., $\delta_0 = 10^{-3}$
    $\text{ind}(t^n,\mathbf{x}) \leftarrow 1$ ▷ Switch to Navier-Stokes solver
  else
    Maintain current indicator
  end if
end if
return $\text{ind}(t^n,\mathbf{x})$

For 1D cases, we similarly define the corresponding vector composed of macroscopic quantities as

$$\mathbf{k}_1^r = \left[\rho^r, u^r, T^r\right]^\mathsf{T} \in \mathbb{R}^3. \tag{3.5}$$

The complete input vector $\mathbf{X}_i$ is then formed by

$$\mathbf{X}_i = \left(\mathbf{k}_1^{i-2}, \mathbf{k}_1^{i-1}, \mathbf{k}_1^i, \mathbf{k}_1^{i+1}, \mathbf{k}_1^{i+2}, \varepsilon, \Delta x\right)^\mathsf{T} \in \mathbb{R}^{17}. \tag{3.6}$$

The label can be defined as

$$Y_i = \begin{cases} 0, & \text{if } I_i \text{ is in the kinetic region,} \\ 1, & \text{if } I_i \text{ is in the Navier-Stokes region,} \\ 2, & \text{if } I_i \text{ is in the Euler region.} \end{cases} \tag{3.7}$$

Algorithm 2 2D training data generator.

Initialize: Computational cell $I_{i,j}$, The initial macroscopic variables ρ, u_x, u_y, T, distribution function f, The final time t_{final}, the time layer t^n
begin the 2D time resolution
for $t^n < t_{\text{final}}$ **do**
    Compute the domain indicator for time t^n by Algorithm 1
    for all the computing cells $I_{i,j}$ **do**
        if $\text{ind}(t^n, \mathbf{x}) = 0$ **then**
            Update by kinetic solver
        else if $\text{ind}(t^n, \mathbf{x}) = 1$ **then**
            Update by Navier-Stokes solver
        end if
        return $\rho^{(i,j)}, u_x^{(i,j)}, u_y^{(i,j)}, T^{(i,j)}, p^{(i,j)}, \varepsilon, \Delta x$
    end for
end for

Table 1: Parameters used to generate the training data for 1D cases.

Parameters	Sod problem	Blast wave problem	Lax problem
Spatial Domain x:	$[-0.2,1.2]$	$[0,1]$	$[-0.2,1.2]$
Spatial Mesh N_x:	50	50	50
Time Domain t:	$[0,0.1]$	$[0,0.1]$	$[0,0.06]$
Total time layers N_t:	289	810	161
Sampling interval t_{interval}:	10	20	5
Knudsen number range:	[5E-04, 1.85E-02]	[5E-04, 1.45E-02]	[5E-04, 4E-03]
Sampling interval $\varepsilon_{\text{interval}}$:	5E-04	5E-04	5E-04

We obtain the dataset based on 1D solvers in [12] and 2D solvers in [13]. In Algorithm 1, we explicitly describe how to collect the indicator values of all cells at time t^n for the 2D cases. The 1D cases can be obtained in a similar manner, only that the fluid solver further distinguishes between the Navier–Stokes and Euler regimes; see [12] for details. The process of 2D data preparation can be seen in Algorithm 2.

Separate datasets are prepared for the 1D and 2D cases. For 1D cases, we generate the macroscopic variables from the Sod problem, the Blast wave problem and the Lax problem. For 2D cases, we generate the macroscopic variables from the Riemann problem and the ghost effect. The parameters used to generate the training data are in Tables 1 and 2.

3.2 Neural network architecture

The macroscopic quantities derived from the training data are fed into a fully connected neural network with a fixed architecture consisting of linear and activation layers. The

Table 2: Parameters used to generate the training data for 2D cases.

Parameters	**Riemann problem**	**Ghost effect**
Spatial Domain x,y:	$[-0.5,0.5]\times[-0.5,0.5]$	$[0,1]\times[0,1]$
Spatial Mesh N_x,N_y:	80×80	40×40
Time Domain t:	$[0.01,0.15]$	$[0.1,1]\cup[2,10]$
Time sampling:	Stride 0.01(15 layers)	• Stride 0.1 in $[0.1,1]$(10 layers) • Stride 1 in $[2,10]$(9 layers)
Knudsen number:	{1E-02, 1E-03}	{1E-02, 3E-02}

mathematical expression for an FCNN can be written as

$$\begin{cases} \mathbf{v}^{H_l}=\sigma^{H_l}(\mathbf{W}^{H_l}\mathbf{v}^{H_{l-1}}+\mathbf{B}^{H_l}), \text{for } l=1,2,3, \\ \mathbf{v}^{\text{out}}=\sigma^{\text{out}}(\mathbf{W}^{\text{out}}\mathbf{v}^{N_L}+\mathbf{B}^{\text{out}}), \\ \widehat{\mathbf{Y}}=\underset{i\in I}{\text{argmax}}\,\mathbf{v}_i^{\text{out}}, I=\{0,1\}, \end{cases} \tag{3.8}$$

where W,B and σ represent the weight matrix, the bias vector, and the activation function, repectively. The output of the network is a probability vector, where each component represents the likelihood that the input corresponds to a specific region. These values range between 0 and 1 and collectively sum to 1. The region associated with the highest probability is selected as the classification result. This final prediction is then used as the region label for the corresponding computational cell in the subsequent equation solving process.

3.3 Training procedure and techniques

In this work, the inputs to the neural network consist of macroscopic quantities such as density, velocity, and temperature. To eliminate discrepancies arising from different physical units and scales, we apply hyperbolic scaling to the BGK equation during the model formulation stage, leading to a fully nondimensionalized system in Eq. (2.1). Specifically, characteristic length, time, and velocity scales are introduced to consistently rescale the spatial, temporal, and velocity variables, as well as the associated macroscopic quantities.

After this nondimensionalization, all macroscopic variables are expressed in dimensionless form and share comparable magnitudes consistent with the intrinsic scales of the kinetic equation. This physics-based scaling approach is standard in kinetic simulations and effectively avoids numerical difficulties caused by large differences in physical units or orders of magnitude.

As a result, no additional statistical normalization or standardization (such as z-score normalization or min–max scaling) is applied before feeding the inputs into the FCNN [33]. The data preprocessing in this study is entirely based on physically consistent nondimensionalization rather than data-driven normalization procedures.

Table 3: Neural Network Training Hyperparameters for 1D and 2D Models.

Parameter	**1D Model**	**2D Model**
Training Samples	126,300	252,800
Train/Test Split (%)	80/20	80/20
Full architecture	[17, 32, 16, 8, 3]	[47, 64, 32, 16, 2]
Activation Functions	ReLU×3, Softmax	ReLU×3, Softmax
Batch size	10,000	10,000
Learning Rate	10^{-4}	10^{-4}
Optimizer	Adam	Adam
Loss Function	L_{CE}	$L_{\text{CE}}+\lambda L_{\text{sym}}$

The training hyperparameters of the neural network are summarized in Table 3. A total of 126,300 data samples were collected for the 1D cases, and 222,400 samples for the 2D cases. The neural network employed in this work consists of three fully connected hidden layers. Specifically, the architecture can be written as [17,32,16,8,3] and [47,64,32,16,2] for 1D cases and 2D cases, respectively. The number of neurons in each hidden layer is chosen to gradually reduce the feature dimension, which helps balance expressive power and computational efficiency while avoiding over-parameterization. Rectified Linear Unit (ReLU) activations are used in the first three hidden layers due to their simplicity, robustness, and favorable gradient propagation properties, which are particularly suitable for fully connected networks. The final layer uses the softmax activation function. Softmax can convert numerical vectors into probability distributions. The mathematical expression for the softmax function is

$$\text{softmax}(z_i)=\frac{\exp(z_i)}{\sum_j \exp(z_j)}, \quad i,j\in\{0,1\}. \tag{3.9}$$

The network is trained using the Adam optimizer with a learning rate of 10^{-3}. Adam is adopted for its adaptive learning rate strategy and stable convergence behavior, especially in problems involving heterogeneous input scales. The batch size of 10,000 is used during training, which provides a good compromise between gradient estimation accuracy and computational efficiency. These hyperparameters are selected based on preliminary numerical experiments and are found to yield stable and reliable training performance across all test cases considered.

We propose a loss function that combines the traditional cross-entropy loss with an additional symmetry constraint to improve the robustness of the FCNN. The input consists of nine units of information arranged in a specific order. To enforce symmetry, we also consider a reversed version of the input sequence by Eq. (3.3). The predictions obtained from the original and reversed inputs are then compared using the Mean Squared Error (MSE) loss. This additional constraint ensures that the model's predictions are consistent under symmetric transformations of the input data. The total loss L is defined as

$$L=L_{\text{CE}}+\lambda L_{\text{sym}}, \tag{3.10}$$

Table 4: Parameters used to test for 1D cases.

Parameters	Sod problem	Blast wave problem	Lax problem
Spatial Domain x:	$[-0.2,1.2]$	$[0,1]$	$[-0.2,1.2]$
Spatial Mesh N_x:	50	50	50
Time Domain t:	$[0.1,0.25]$	$[0.1,0.25]$	$[0.06,0.12]$
Knudsen number range:	[5E-04, 1.85E-02]	[5E-04, 1.45E-02]	[5E-04, 4E-03]

Table 5: Parameters used to test for 2D cases.

Parameters	Riemann problem	Ghost effect
Spatial Domain x,y:	$[-0.5,0.5]\times[-0.5,0.5]$	$[0,1]\times[0,1]$
Spatial Mesh N_x,N_y:	80×80	40×40
Time Domain t:	$[0.15,0.35]$	$[10,20]$
Knudsen number range:	[1E-04, 1E-02]	[1E-02, 3E-02]

where L_{CE} is the cross-entropy loss and L_{sym} is the symmetry loss defined as the MSE between the predictions of the original output $\widehat{Y}_{i,j}$ and the reversed output $\widehat{Y}^{\text{rev}}_{i,j}$. Mathematically, they can be written as

$$\begin{cases} L_{\text{CE}} = -\dfrac{1}{N_{\text{train}}}\displaystyle\sum_{i,j}\widehat{Y}_{i,j}\log(p_{i,j})+(1-\widehat{Y}_{i,j})\log(1-p_{i,j}) \\ L_{\text{sym}} = \dfrac{1}{N_{\text{train}}}\displaystyle\sum_{i,j}(\widehat{Y}_{i,j}-\widehat{Y}^{\text{rev}}_{i,j}), \end{cases} \tag{3.11}$$

where N_{train} is the number of training samples and $p_{i,j}$ is the predicted probability that $\widehat{Y}_{i,j}$ belongs to the positive class (i.e., label 1). $\widehat{Y}_{i,j},\widehat{Y}^{\text{rev}}_{i,j}$ are the outputs for input data $\mathbf{X}_{i,j}$ and $\mathbf{X}^{\text{rev}}_{i,j}$, respectively. λ is a hyperparameter that controls the weight of the symmetry constraint. Here we take $\lambda=1$. Finally we obtain the weights $\mathbf{W}^{H_1},\mathbf{W}^{H_2},\mathbf{W}^{H_3},\mathbf{W}^{\text{out}}$ and the biases $\mathbf{B}^{H_1},\mathbf{B}^{H_2},\mathbf{B}^{H_3},\mathbf{B}^{\text{out}}$ after training. Then we replace the moment realizability matrix-based domain decomposition method with the FCNN model.

4 Numerical results

In this section, we will test the solution of 1D and 2D BGK equations with indicators obtained by FCNN. The parameters for the test cases are listed in Tables 4 and 5. Our kinetic and hydrodynamic solvers are refered to [12] for the 1D case as well as [13] for the 2D case. For the 1D case, we use a third order nodal discontinuous Galerkin (NDG3) finite element approach for spatial discretization as well as third order implicit–explicit (IMEX) Runge–Kutta (RK) method for time discretization. For the 2D case, The first order NDG approach and the first order IMEX method are applied.

We denote the neural network-based domain decomposition method as NN-Decomp, and the moment realizability matrix-based method as Original-RM. The formula for the

Table 6: Comparison of relative L^2 errors for 2D Riemann problem, $\varepsilon=10^{-3}$.

Time	NN-Decomp			Original-RM		
	ρ	p	T	ρ	p	T
t=0.10	1.91e-04	1.80e-04	7.55e-04	6.45e-04	7.05e-04	2.60e-03
t=0.20	1.54e-04	1.85e-04	6.30e-04	5.86e-04	7.81e-04	2.57e-03
t=0.25	1.66e-04	2.31e-04	6.91e-04	9.51e-04	1.15e-03	2.73e-03

Table 7: Comparison of relative errors for the 1D Sod problem between NN-Decomp and Original-RM methods (with reference to Kinetic solution).

ε	N_x	Time	NN-Decomp			Original-RM		
			ρ	u	T	ρ	u	T
0.00579	40	t=0.15	2.58e-03	2.18e-02	1.72e-02	3.03e-03	1.67e-02	9.34e-03
		t=0.20	3.54e-03	2.58e-02	2.37e-02	3.68e-03	1.71e-02	1.24e-02
		t=0.25	4.74e-03	3.81e-02	3.03e-02	4.51e-03	2.07e-02	1.68e-02
	60	t=0.15	4.19e-03	3.75e-02	2.17e-02	4.46e-03	3.60e-02	1.85e-02
		t=0.20	4.90e-03	3.47e-02	2.19e-02	4.64e-03	3.06e-02	1.79e-02
		t=0.25	5.88e-03	3.21e-02	2.22e-02	4.94e-03	2.70e-02	1.65e-02
0.01246	40	t=0.15	2.75e-03	3.02e-02	2.13e-02	2.85e-03	2.87e-02	1.68e-02
		t=0.20	2.77e-03	2.83e-02	2.26e-02	3.08e-03	2.26e-02	1.60e-02
		t=0.25	3.32e-03	2.87e-02	2.25e-02	3.65e-03	2.17e-02	1.48e-02
	60	t=0.15	3.35e-03	2.76e-02	1.93e-02	3.54e-03	3.11e-02	2.14e-02
		t=0.20	3.87e-03	2.75e-02	1.97e-02	3.59e-03	2.96e-02	2.11e-02
		t=0.25	4.75e-03	2.67e-02	1.99e-02	4.08e-03	2.73e-02	2.06e-02

Table 8: Comparison of CPU time and relative errors for the 1D Blast wave problem between NN-Decomp and Original-RM methods(with reference to Kinetic solution, $\varepsilon=0.00766$).

Time	NN-Decomp				Original-RM				Kinetic
	ρ	u	T	CPU(s)	ρ	u	T	CPU(s)	CPU(s)
t=0.10	1.46%	4.42%	3.77%	57.70	1.62%	4.72%	3.87%	54.85	107.15
t=0.20	2.04%	13.2%	4.13%	103.36	1.94%	14.1%	4.18%	125.17	213.00
t=0.25	1.78%	7.28%	4.43%	130.64	1.88%	9.04%	5.06%	157.92	281.81

relative L^2 error is given in Eq. (4.1)

$$E_2=\sqrt{\frac{\sum_{i,j}(u_{\text{appx}}(x_i,y_j,t)-u_{\text{ref}}(x_i,y_j,t))^2}{\sum_{i,j}(u_{\text{ref}}(x_i,y_j,t))^2}}. \tag{4.1}$$

Taking the full kinetic solver as the reference solution, we compute the relative L^2 errors for both the NN-Decomp and Original-RM domain decomposition methods in Tables 6-8. To further assess the generalization capability of the proposed NN-Decomp

method, we additionally consider the 1D Sod problem with Knudsen numbers that are not included in the training set, and report the corresponding relative errors at different times and spatial resolutions. The results are summarized in Table 7. This test is designed to examine the robustness of the trained neural network when applied to unseen Knudsen numbers and different mesh resolutions, while keeping the network parameters fixed. We also compare the CPU time and relative errors of the 1D blast wave problem between NN-Decomp and Original-RM methods at different times($\varepsilon = 0.00766$). To ensure the reliability of the results, the CPU time for each set of ε values is measured three times and averaged. The results are summarized in Table 8. Through numerical tests, we observe that our FCNN domain indicators can have a good effect on the long-term evolution of the BGK equation.

Example 4.1 (1D Sod problem)**.** First we consider the 1D Sod shock tube problem with initial conditions to be

$$(\rho,u,T) = \begin{cases} (1,0,1) & \text{if } x < 0.5; \\ (0.125,0,0.8) & \text{if } x > 0.5, \end{cases} \tag{4.2}$$

on the domain $[-0.2,1.2]$, $v \in [-4.5,4.5]$.

We present the numerical results for $\varepsilon = 10^{-2}$ in Fig. 3 and for $\varepsilon = 10^{-3}$ in Fig. 4, using the FCNN-based domain indicators. As expected, kinetic regions are mainly identified in areas where the macroscopic quantities (ρ,u,T) exhibit sharp spatial variations. Meanwhile, the Knudsen number also plays an important role in the resulting domain decomposition. For $\varepsilon = 10^{-2}$, all three regimes (Euler, Navier–Stokes, and kinetic) coexist, whereas for $\varepsilon = 10^{-3}$ the solution is predominantly governed by the Navier–Stokes and Euler solvers.

To further quantitatively assess the generalization capability of the proposed NN-Decomp method, we report in Table 7 the relative L^2 errors of density, velocity, and temperature for the 1D Sod problem at different Knudsen numbers, spatial resolutions, and output times, with respect to the full kinetic solution. Notably, the Knudsen numbers considered in Table 7 are not included in the training set, and the tests are conducted on different spatial grids. The results show that NN-Decomp consistently achieves comparable or smaller errors than the Original-RM method across all configurations, indicating that the trained FCNN domain indicators can generalize well to unseen Knudsen numbers and different spatial resolutions, and remain effective during the long-term evolution of the BGK equation.

Example 4.2 (1D blast wave problem)**.** For the 1D blast wave problem, the initial condition is given by

$$(\rho,u,T) = \begin{cases} (1,1,2) & \text{if } x < 0.2; \\ (1,0,0.25) & \text{if } 0.2 \le x \le 0.8; \\ (1,-1,2) & \text{if } x > 0.8, \end{cases} \tag{4.3}$$

with reflective boundary condition in the x direction on the domain $[0,1]$ and $v \in [-9,9]$.

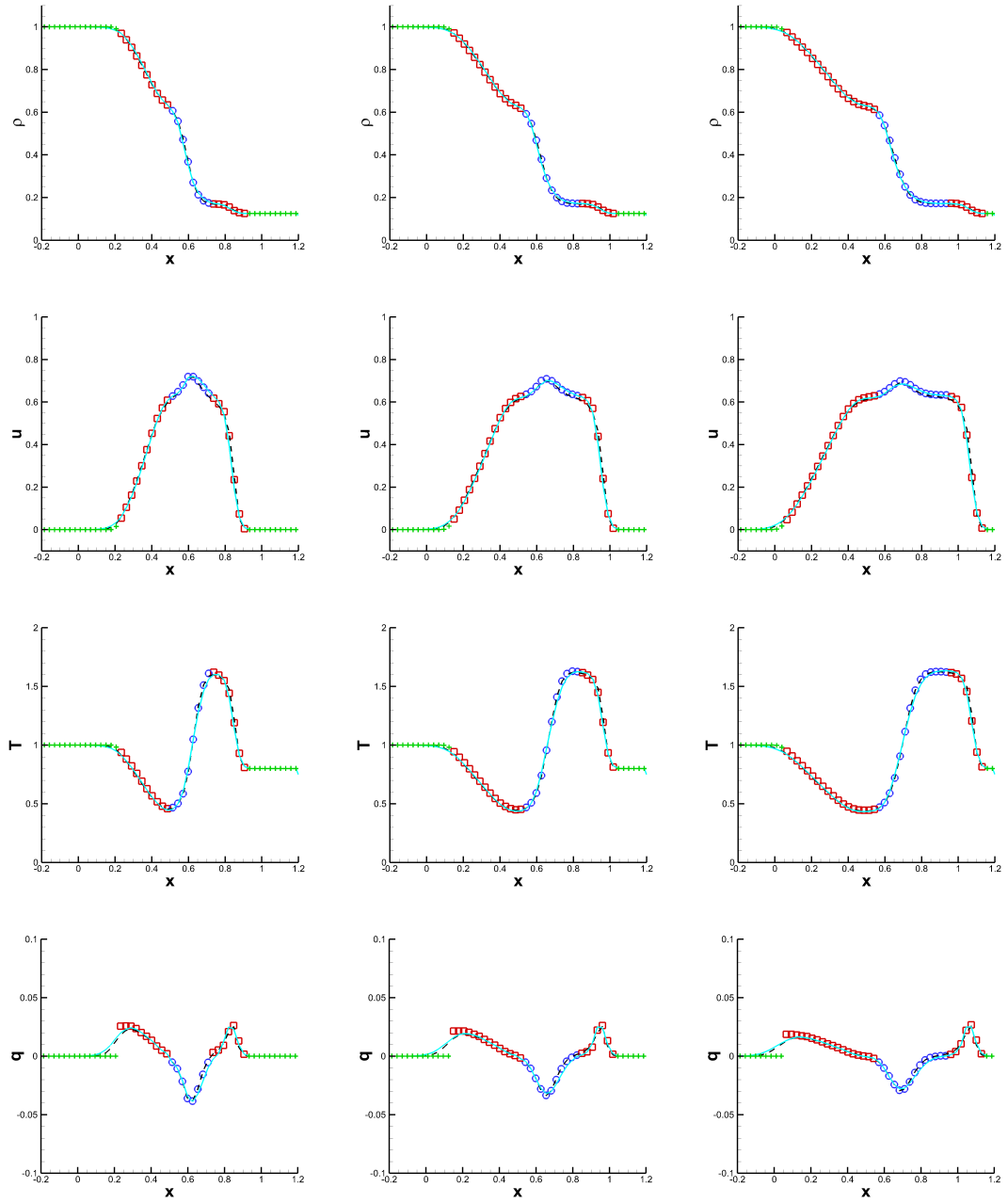

Figure 3: 1D Sod problem on the domain $[-0.2,1.2]\times[-4.5,4.5]$. Symbols: $N_x=50$ and $N_v=100$ with NDG3. From left to right: time $t=0.15,0.2,0.25$. From top to bottom, the density ρ, the mean velocity u, the temperature T and the heat flux q. Symbols: "+" is Euler, square is NS, circle is kinetic. Solid line: the NS reference solution. Dashed line: the kinetic reference solution. $\varepsilon=10^{-2}$, FCNN domain indicators.

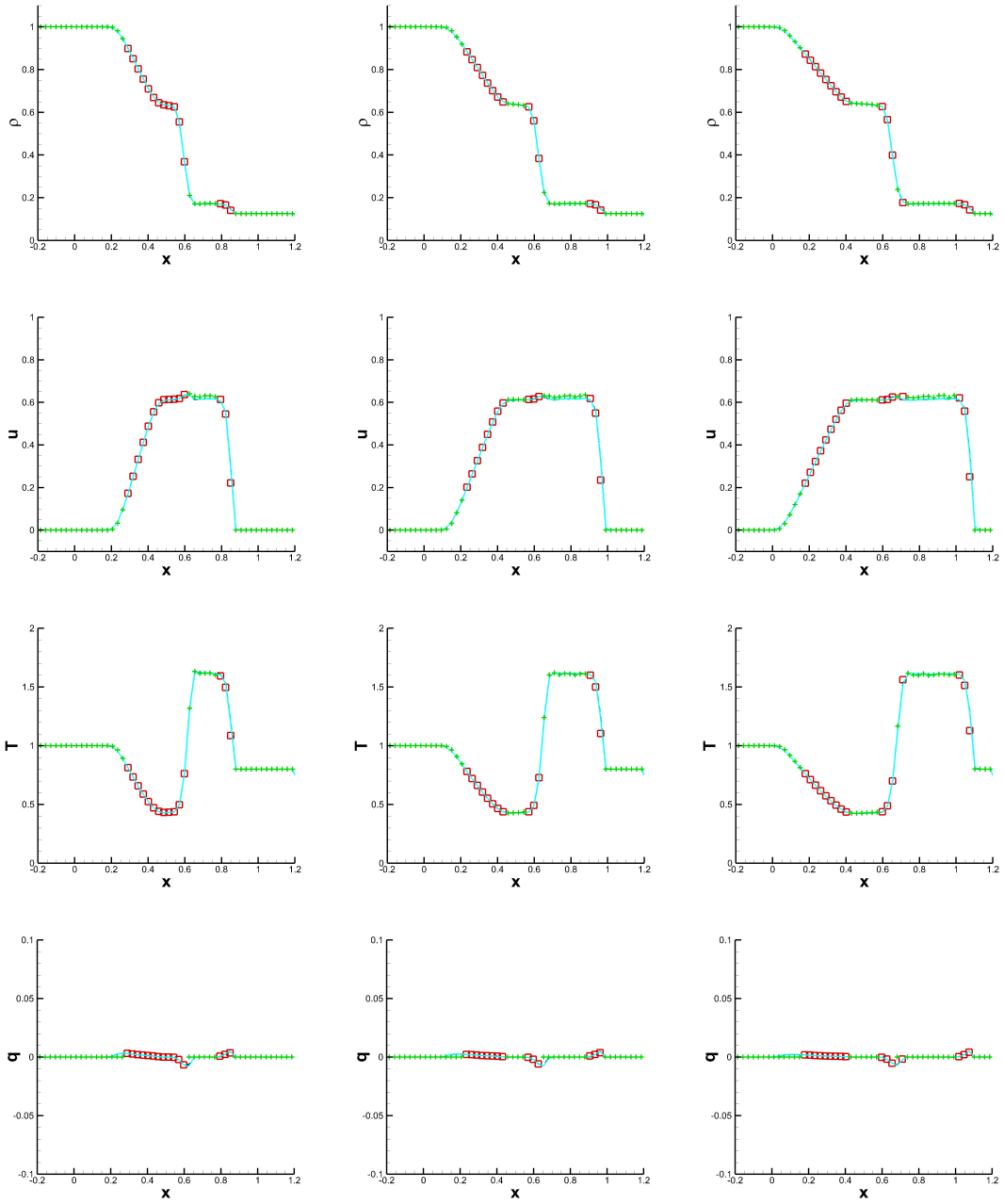

Figure 4: 1D Sod problem on the domain $[-0.2,1.2]\times[-4.5,4.5]$. Symbols: $N_x=50$ and $N_v=100$ with NDG3. From left to right: time $t=0.15,0.2,0.25$. From top to bottom, the density ρ, the mean velocity u, the temperature T and the heat flux q. Symbols: "+" is Euler, square is NS, circle is kinetic. Solid line: the NS reference solution. Dashed line: the kinetic reference solution. $\varepsilon=10^{-3}$, FCNN domain indicators.

Similarly, we show the results for $\varepsilon = 10^{-2}$ in Fig. 5 and for $\varepsilon = 10^{-3}$ in Fig. 6, by FCNN domain indicators. We can see the figures in the fourth line in case $\varepsilon = 10^{-2}$, the kinetic solver differs from the Navier-Stokes solver, while our FCNN domain indicators accurately identify regions of difference as kinetic regions.

To assess the computational efficiency of the proposed NN-Decomp method, we compare its total CPU time with that of the Original-RM method for the 1D blast wave problem. The results are summarized in Table 8. Both methods are evaluated at different time instances, and the kinetic solution is used as a reference.

As shown in Table 8, the NN-Decomp method achieves accuracy comparable to the Original-RM method in terms of the relative errors of density, velocity, and temperature. Meanwhile, it exhibits a clear reduction in computational cost at later times. Specifically, at $t = 0.20$ and $t = 0.25$, the total CPU time is reduced by approximately 17% compared with the Original-RM approach.

The efficiency gain mainly stems from the avoidance of repeated spatial derivative calculations and eigenvalue decompositions required by the realizability-matrix-based indicator. Instead, the neural network directly evaluates the domain decomposition indicator using macroscopic quantities at a much lower cost. These results provide clear quantitative evidence that the NN-Decomp method improves computational efficiency while preserving accuracy.

Example 4.3 (1D Lax problem)**.** For the 1D Lax problem, the initial condition is given by

$$(\rho,u,T) = \begin{cases} (0.445,0.698,7.928) & \text{if } x < 0.5; \\ (0.5,0,1.142) & \text{if } x > 0.5, \end{cases} \tag{4.4}$$

on the domain $[-0.2,1.2]$, $v \in [-4.5,4.5]$.

We show the results for $\varepsilon = 4\times 10^{-3}$ in Fig. 7 and for $\varepsilon = 10^{-3}$ in Fig. 8. It is worth noting that [12] does not include a numerical example for the Lax problem. Through testing, we found that directly applying the manually chosen parameters from [12] for domain decomposition leads to significant discrepancies between the numerical solution and the kinetic reference solution. This may be attributed to the fact that the Lax problem generates stronger shock waves and steeper gradients, which exceed the applicability of the standard Navier–Stokes equations. More importantly, this observation highlights a fundamental limitation of realizability-matrix-based domain decomposition methods: their performance strongly depends on problem-specific manual parameters. Parameters that work well for one flow configuration may fail when applied to a different regime with stronger non-equilibrium effects. This lack of robustness and generalizability serves as a primary motivation for introducing a data-driven, auto-tuning alternative.

Therefore, we re-tuned the manual parameter η_0,η_1,δ_0 (defined in [12]) and found that $\eta_0 = 10^{-3},\eta_1 = 10^{-4},\delta_0 = 10^{-5}$ yields improved performance for domain decomposition in this case. Based on this re-tuned value, we generated new training data for the Lax problem. Figs. 7-8 present a comparison of numerical solutions: the left panel shows the result obtained via neural network-based domain decomposition, named NN-Decomp; the

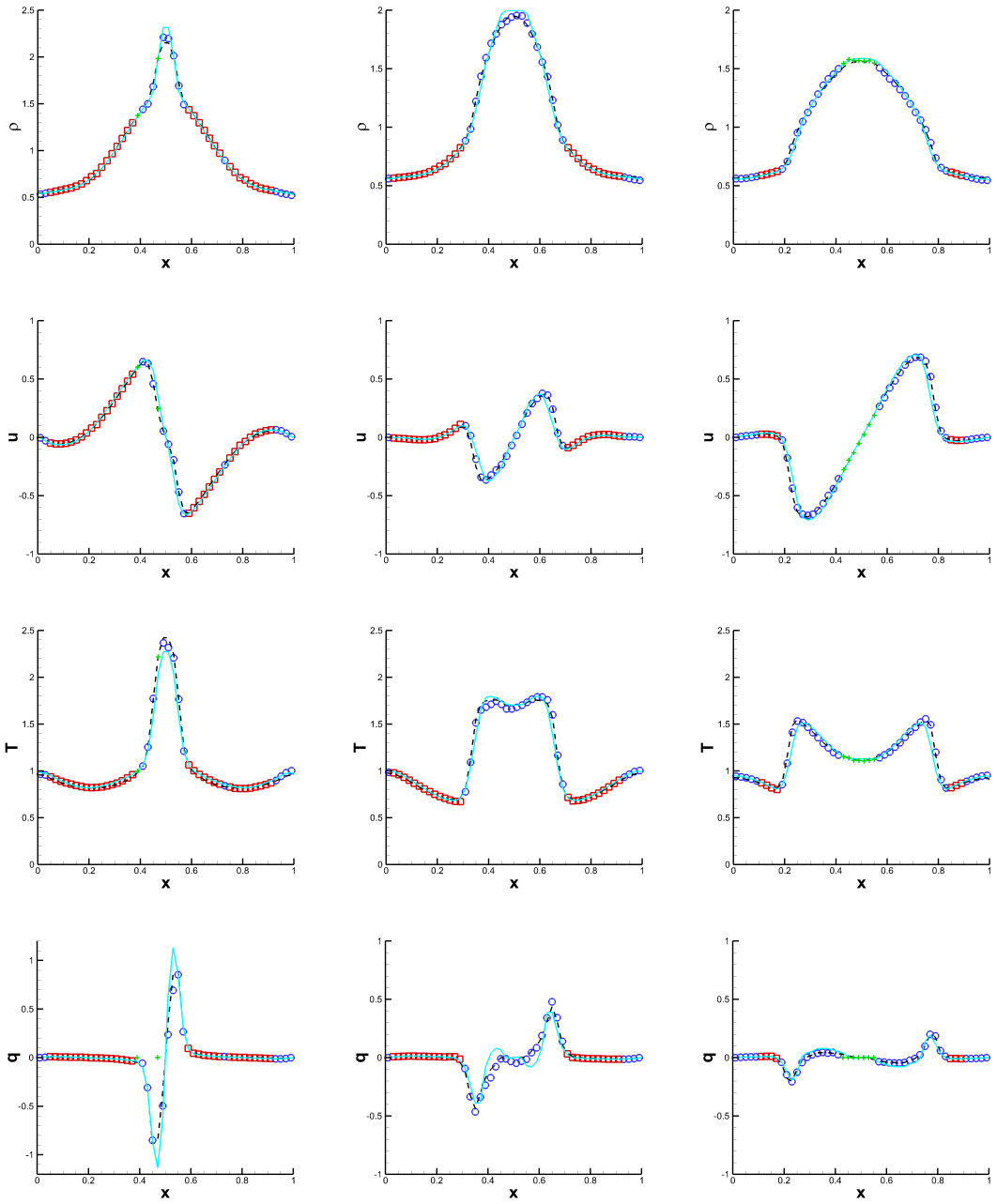

Figure 5: 1D blast wave problem on the domain $[0,1]\times[-9,9]$. Symbols: N_x=50 and N_v=100 with NDG3. From left to right: time t=0.15,0.2,0.25. From top to bottom, the density ρ, the mean velocity u, the temperature T and the heat flux q. Symbols: "+" is Euler, square is NS, circle is kinetic. Solid line: the NS reference solution. Dashed line: the kinetic reference solution. $\varepsilon=10^{-2}$, FCNN domain indicators.

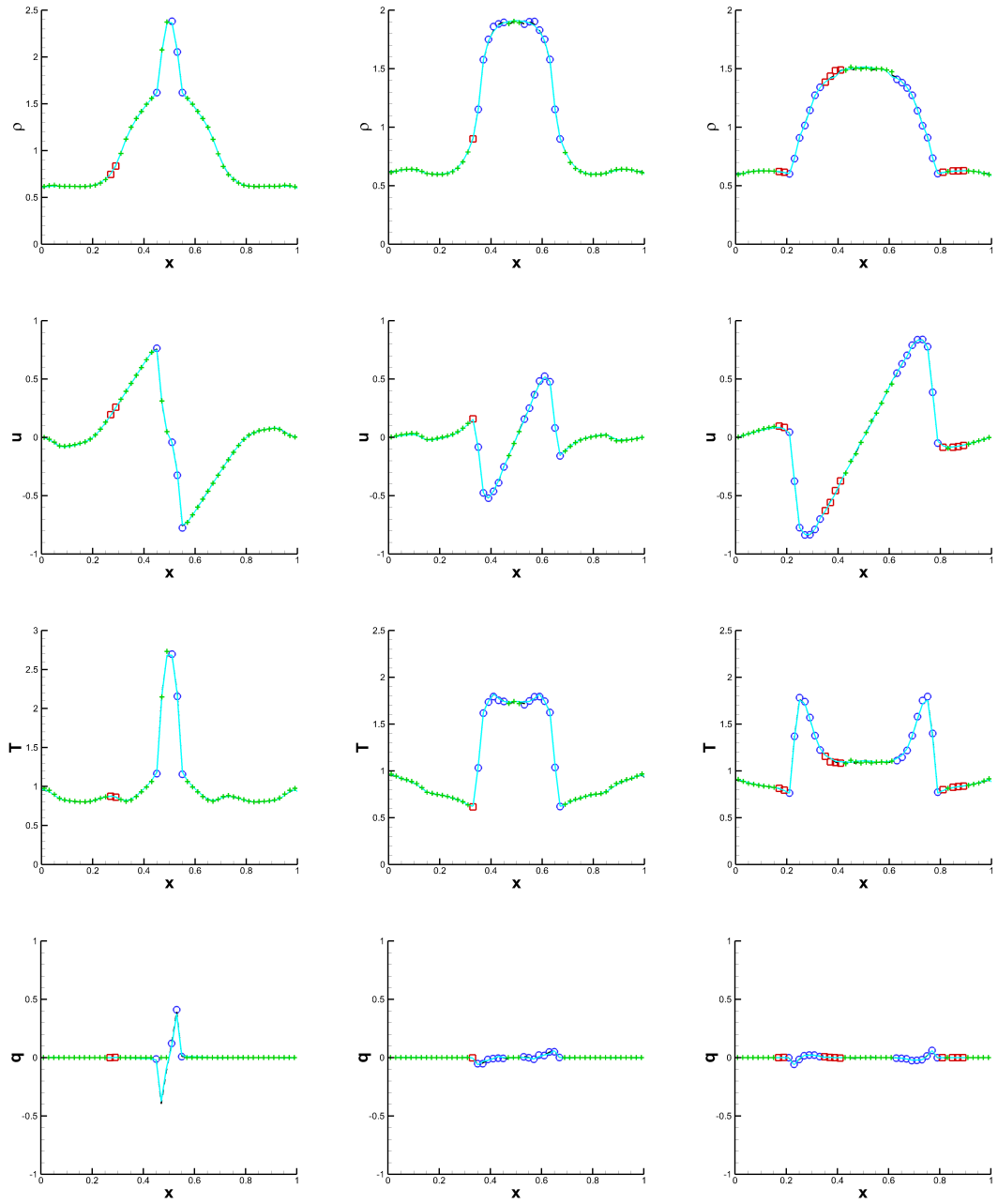

Figure 6: 1D blast wave problem on the domain $[0,1]\times[-9,9]$. Symbols: N_x=50 and N_v=100 with NDG3. From left to right: time t=0.15,0.2,0.25. From top to bottom, the density ρ, the mean velocity u, the temperature T and the heat flux q. Symbols: "+" is Euler, square is NS, circle is kinetic. Solid line: the NS reference solution. Dashed line: the kinetic reference solution. $\varepsilon=10^{-3}$, FCNN domain indicators.

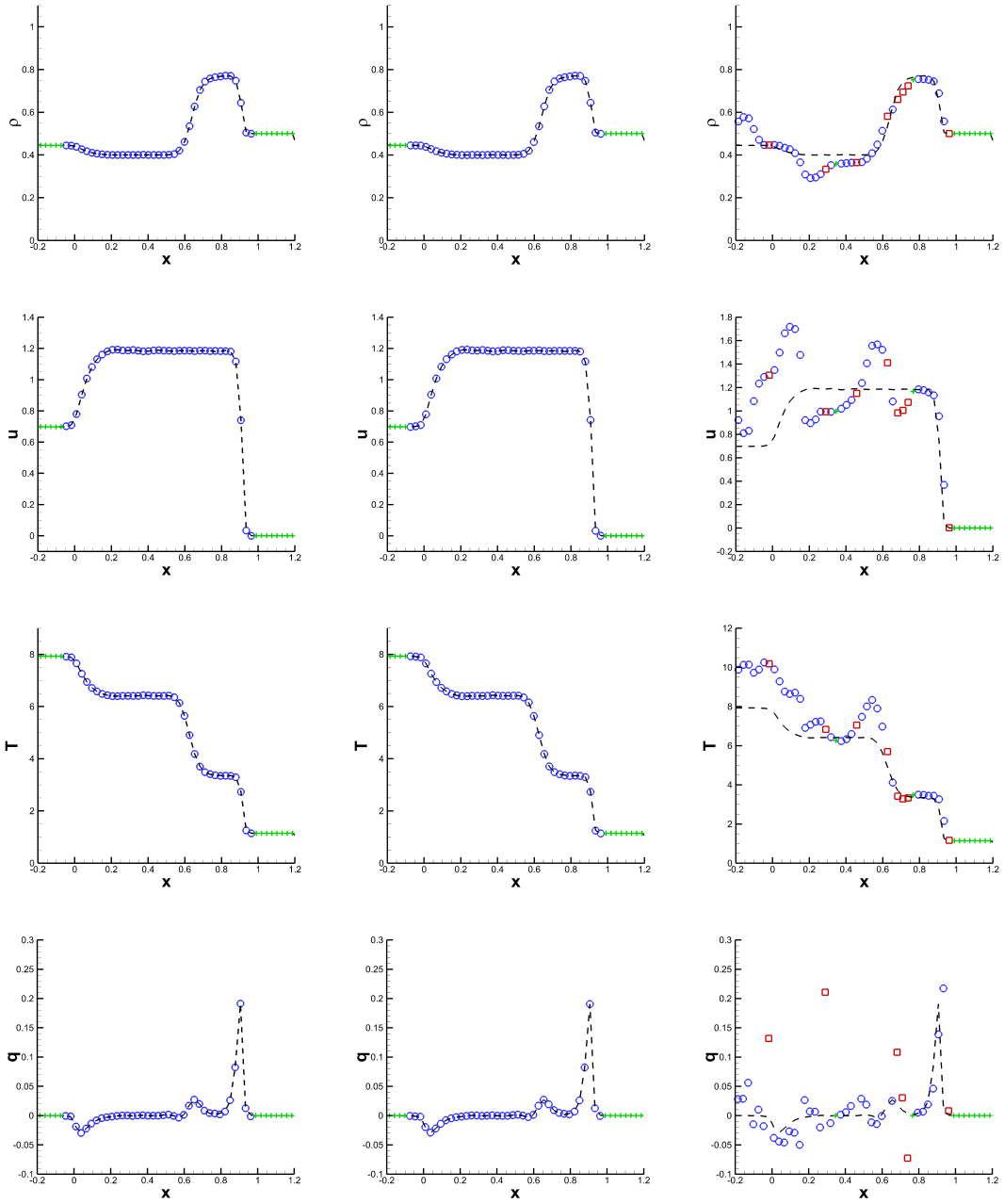

Figure 7: 1D Lax problem on the domain $[-0.2,1.2]\times[-4.5,4.5]$. Symbols: $N_x=50$ and $N_v=100$ with NDG3. From left to right: NN-Decomp, Tuned-RM, Untuned-RM. From top to bottom, the density ρ, the mean velocity u, the temperature T and the heat flux q. Symbols: "+" is Euler, square is NS, circle is kinetic. Solid line: the NS reference solution. Dashed line: the kinetic reference solution. $\varepsilon=4\times10^{-3}$,$t=0.12$.

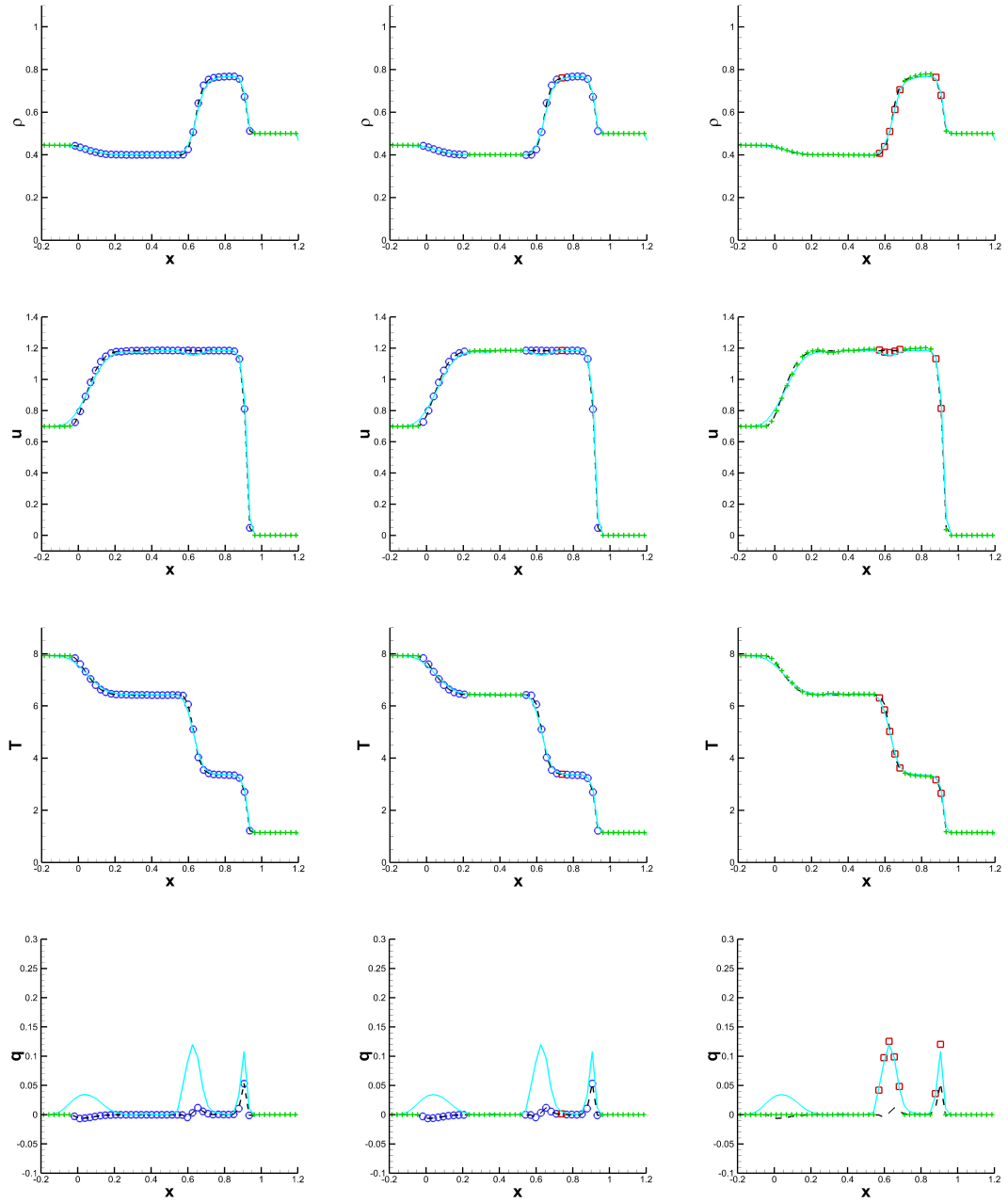

Figure 8: 1D Lax problem on the domain $[-0.2,1.2]\times[-4.5,4.5]$. Symbols: $N_x=50$ and $N_v=100$ with NDG3. From left to right: NN-Decomp, Tuned-RM, Original-RM. From top to bottom, the density ρ, the mean velocity u, the temperature T and the heat flux q. Symbols: "+" is Euler, square is NS, circle is kinetic. Solid line: the NS reference solution. Dashed line: the kinetic reference solution. $\varepsilon=10^{-3}$,$t=0.12$.

middle panel shows the solution obtained using the realizability matrix-based method with the re-tuned parameter, named Tuned-RM; and the right panel shows the solution using the original, unmodified parameter, named Original-RM.

The results demonstrate that the realizability-matrix-based method is highly sensitive to manually chosen parameters, which limits its robustness across different problems. In contrast, the proposed NN-Decomp method avoids manual tuning and consistently produces solutions that closely match the kinetic reference, providing a more reliable and problem-independent alternative.

Example 4.4 (2D Riemann problem)**.** In this example, we now consider a 2D Riemann problem for polytropic gas with initial datum in four quadrants

$$(\rho,p,u,v)(x,y,0)=\begin{cases}(1.5,1.5,0,0), & \text{if } x\geq 0 \text{ and } y\geq 0,\\ (0.6429,0.3,1.0328,0), & \text{if } x\leq 0 \text{ and } y\geq 0,\\ (0.1891,0.0143,1.0328,1.0328), & \text{if } x\leq 0 \text{ and } y\leq 0,\\ (0.6429,0.3,0,1.0328), & \text{if } x\geq 0 \text{ and } y\leq 0.\end{cases} \tag{4.5}$$

Here we have $\gamma=5/3$.

To avoid numerical oscillations in the shock problem (which typically require limiters in high-order DG schemes), we adopt a first-order discontinuous Galerkin discretization along both x and y directions. The mesh is uniform with $N_x=N_y=80$, and the velocity space is truncated to $\Omega_v=[-8,8]$ with $N_v=64$ in each direction. The C++ solver is parallelized along the y-direction using 4 processors. The NN-Decomp indicators produce regions that are generally contiguous and closely match the full kinetic solution.

We further evaluate the effect of including a symmetry loss term in the training of the NN-Decomp model in Figs. 9 and 10. Specifically, we compare three setups: (i) NN-Decomp trained with symmetry loss, (ii) NN-Decomp trained without symmetry loss, and (iii) Original-RM. We observe that the model trained without symmetry loss produces asymmetric domain indicators as the simulation evolves in time. This asymmetry can introduce inconsistencies in the solver and potentially degrade the solution quality. In contrast, the model trained with symmetry loss maintains spatially consistent and symmetric indicators, leading to more robust and accurate results.

Figs. 11 and 12 show the macroscopic quantities for $\varepsilon=10^{-2}$ and $\varepsilon=10^{-3}$, respectively. Table 6 presents the corresponding relative L^2 errors for $\varepsilon=10^{-3}$. The results indicate that NN-Decomp consistently yields lower errors than the Original-RM approach, demonstrating its superior accuracy across all macroscopic variables.

Example 4.5 (2D ghost effect)**.** We consider a rarefied gas between two parallel plane walls at $y=0$ and $y=1$. The walls both have a common periodic temperature distribution T_w

$$T_w(x)=1-0.5\cos(2\pi x),\forall x\in(0,1),$$

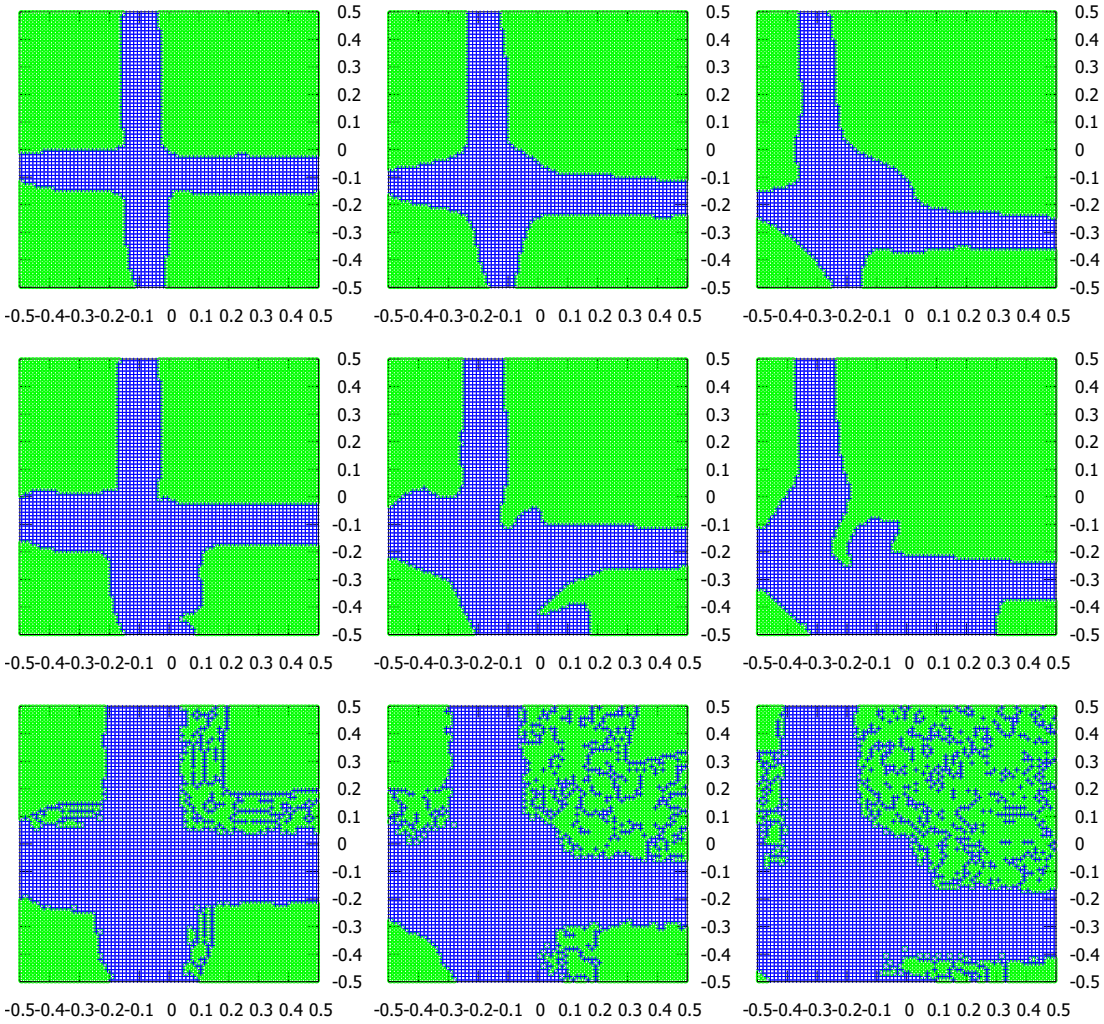

Figure 9: **2D Riemann problem**. Domain indicators profile obtained with a first order discontinuous Galerkin scheme with uniform grids with $N_x=N_y=80$ and $\varepsilon=10^{-2}$. From left to right: t=0.1,0.2,0.35. Top: NN-Decomp with symmetry loss; middle: NN-Decomp without symmetry loss; bottom: Original-RM. Symbol "+" denotes kinetic cells, symbol "o" denotes hydrodynamic cells.

and move in a common small mean velocity u_w of order ε

$$u_w(x)=(\varepsilon,0).$$

We take a uniform mesh with $N_x=N_y=40$ and time step $\Delta t=1/5000$. For velocity, since the solution is smooth and does not vary too much, we take a cut-off domain $\Omega_v=[-8,8]$ and discretize it with $N_v=16$ points along each direction. We apply a first order nodal discontinuous Galerkin scheme and force the grids inside the width of 0.1 along x direction around the walls always to be kinetic. We show the results for $\varepsilon=0.02, t=20$ in

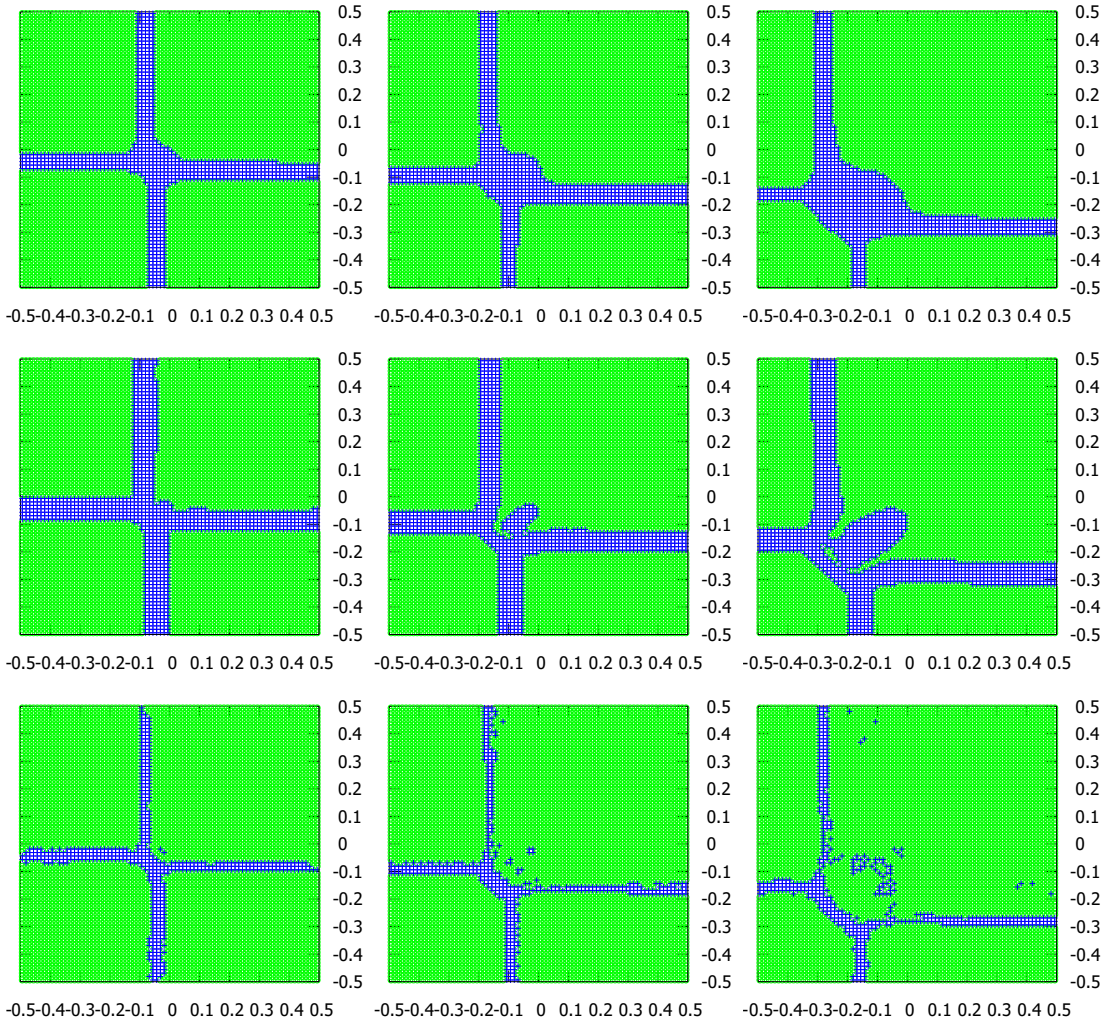

Figure 10: **2D Riemann problem**. Domain indicators profile obtained with a first order discontinuous Galerkin scheme with uniform grids with $N_x=N_y=80$ and $\varepsilon=10^{-3}$. From left to right: $t=0.1,0.2,0.35$. Top: NN-Decomp with symmetry loss; middle: NN-Decomp without symmetry loss; bottom: Original-RM. Symbol "+" denotes kinetic cells, symbol "o" denotes hydrodynamic cells.

Fig. 13. We observe that the areas divided by our FCNN indicators are mostly connected, which can reduce the interface coupling.

4.1 Error analysis and robustness

Misclassification by the FCNN may occur, especially near the transition between kinetic and fluid regimes. In our simulations, such misclassification is typically localized and limited to a small number of cells. These cells usually lie in regions where the kinetic and

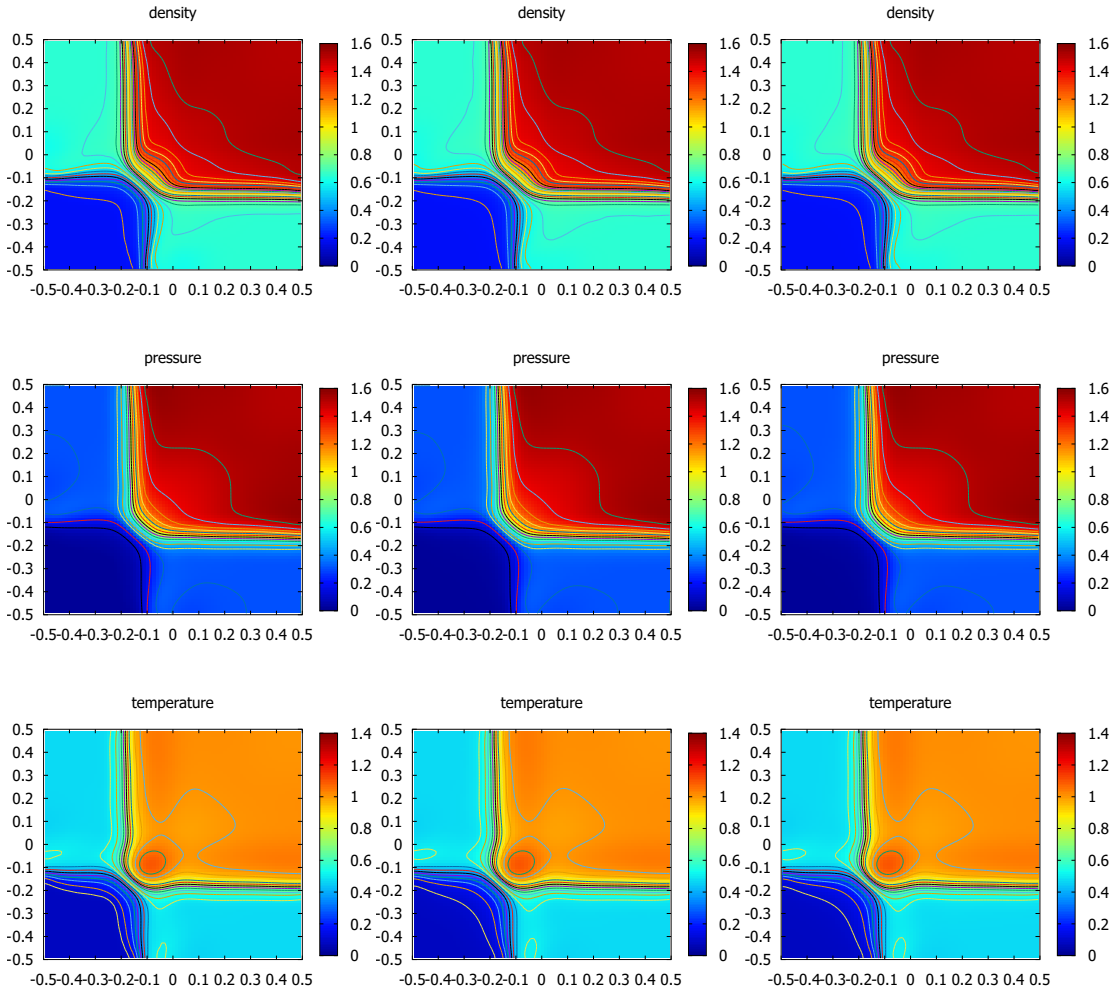

Figure 11: **2D Riemann problem**. Macroscopic quantities profile obtained with a first order discontinuous Galerkin scheme using uniform grids with $N_x = N_y = 80$ and $\varepsilon = 10^{-2}$. From left to right: NN-Decomp with symmetry loss, Original-RM, full kinetic model. From top to bottom: the density, the pressure and the temperature. $t = 0.2$, 29 contour lines.

fluid solutions are already close.

When a kinetic region is misclassified as fluid, the main effect is a local underestimation of non-equilibrium effects. This may lead to a slightly smoother solution in that region. However, the impact remains local and does not propagate to the entire domain. We do not observe numerical instability or noticeable degradation of the macroscopic quantities.

Overall, the proposed NN-Decomp method shows good robustness with respect to occasional classification errors, while maintaining stable and accurate solutions.

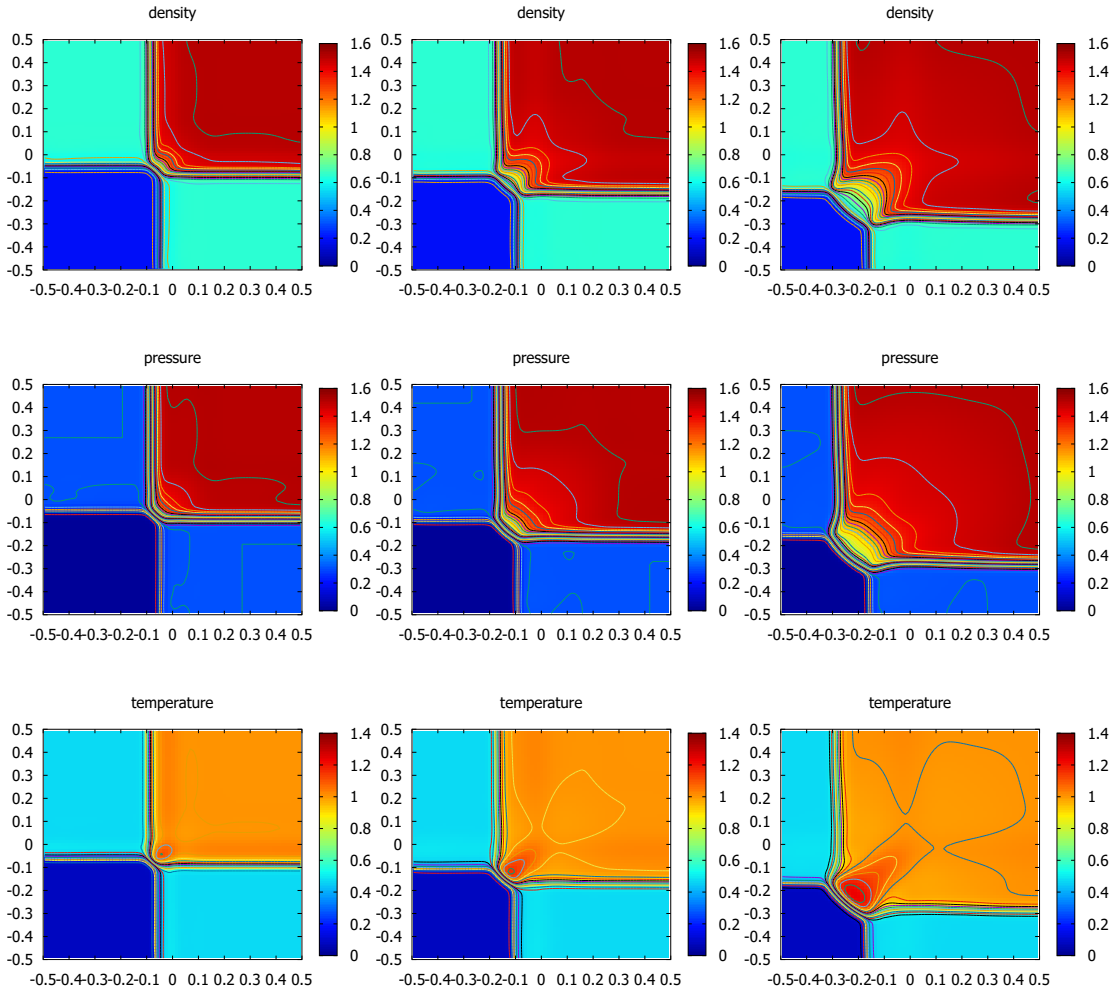

Figure 12: **2D Riemann problem**. Macroscopic quantities profile obtained with a first order discontinuous Galerkin scheme using uniform grids with $N_x=N_y=80$ and $\varepsilon=10^{-3}$. From left to right: $t=0.1,0.2,0.35$. From top to bottom: the density, the pressure and the temperature. NN-Decomp with symmetry loss, 29 contour lines on the range $[0,1.6]$.

5 Conclusion

In this paper, we propose an FCNN-based alternative to the domain decomposition method based on the moment realizability matrix. The network is trained using solution data and corresponding labels collected over a short time interval, after which it is integrated into the full simulation. For the long-time evolution of the BGK equation, the FCNN-based indicator produces results that remain close to those of the full kinetic solver.

Compared with the realizability-matrix-based approach, the proposed NN-Decomp

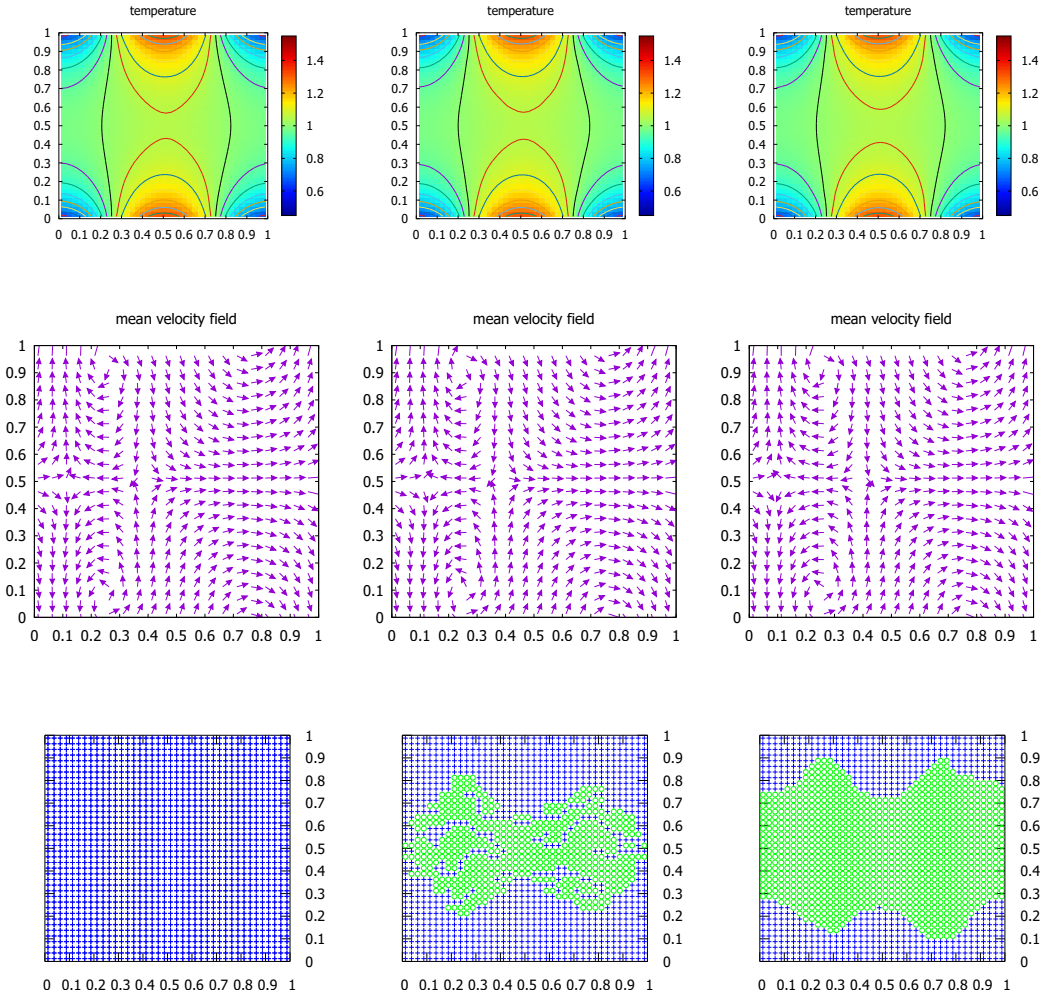

Figure 13: **The ghost effect**. First order discontinuous Galerkin scheme. Uniform grids with $N_x = N_y = 40$. $\varepsilon = 0.02$. $t = 20$. From left to right: the full kinetic model, the Original-RM, the NN-Decomp. From top to bottom: the isothermal lines, the mean velocity field and the domain indicator. In the domain indicator, symbol "+" denotes kinetic cells, and symbol "o" denotes hydrodynamic cells. The isothermal lines increase by 0.05 on the range $[0.45, 1.55]$.

method eliminates the need for manual parameter tuning and provides a direct evaluation of the domain decomposition indicator. Moreover, the resulting kinetic and fluid regions are generally connected, which reduces the computational overhead associated with interface coupling.

The performance of the NN-Decomp method depends on the representativeness of the training data. In this work, the network is trained on a range of flow regimes and Knudsen numbers relevant to the considered test problems, and it demonstrates good

accuracy and robustness when applied to problems with similar physical characteristics. For problems with fundamentally different physics, re-training or fine-tuning of the network is expected. This limitation is common to data-driven approaches and does not affect the consistency of the underlying hybrid kinetic–fluid framework.

Future work will focus on improving the generalizability of the proposed method, including the investigation of transfer learning and online adaptation strategies. In addition, we plan to explore further applications of the FCNN-based indicator in more complex kinetic–fluid coupling problems.

Acknowledgments

This work is partially supported by the National Key R&D Program of China No. 2022YFA1004500, NSFC grant Nos. 92270112 and 12571443, and NSF of Fujian Province No. 2023J02003.

A Hydrodynamic solvers

We follow [13] and [12] to establish the formulation of hydrodynamic solvers.

A.1 Preliminary

For 2D BGK equation, we set a partition to be a rectangular $\Omega_x=[a,b]\times[c,d]$, $a=x_{\frac{1}{2}}<x_{\frac{3}{2}}<\cdots<x_{N_x+\frac{1}{2}}=b, c=y_{\frac{1}{2}}<y_{\frac{3}{2}}<\cdots<y_{N_y+\frac{1}{2}}=d$. Let $I_{i,j}=\left[x_{i-\frac{1}{2}},x_{i+\frac{1}{2}}\right]\times\left[y_{i-\frac{1}{2}},y_{i+\frac{1}{2}}\right]$ denotes an element with its length $\Delta x_i=x_{i+\frac{1}{2}}-x_{i-\frac{1}{2}}$ and $\Delta y_j=y_{j+\frac{1}{2}}-y_{j-\frac{1}{2}}$. Set $h_x=\max_{i=1}^{N_x}\Delta x_i, h_y=\max_{j=1}^{N_y}\Delta y_j$ which represent the maximum of the spatial interval. Note $h=\max(h_x,h_y)$. Given any non-negative integer vector $\mathbf{K}=(K_1,K_2)$, we define a finite dimensional discrete space,

$$\mathcal{Z}_h^{\mathbf{K}}=\{w|w\in L^2(\Omega_{\mathbf{x}}),w|_{I_{i,j}}\in Q^{\mathbf{K}}(I_{i,j}),1\le i\le N_x,1\le j\le N_y\}. \tag{A.1}$$

and its vector version is denoted as

$$\mathbf{Z}_h^{\mathbf{K}}=\{\mathbf{w}=(w_1,w_2,w_3,w_4)^T|w_l\in\mathcal{Z}_h^{\mathbf{K}},1\le l\le 4\}. \tag{A.2}$$

We take the 2D local space $Q^{\mathbf{K}}(I_{i,j})$ as a tensor product space of $P^{K_1}(I_i)\otimes P^{K_2}(I_j)$, where the degree of polynomials in local space $P^K(I)$ are at most K on I. Obviously, the functions in $\mathcal{Z}_h^{\mathbf{K}}$ are piecewise defined. We declare that the left and right limits of a function $u\in\mathcal{Z}_h^{\mathbf{K}}$ at the interface $(x_{i+\frac{1}{2}},y)$ along y direction are denoted as

$$u(x_{i+\frac{1}{2}}^{\pm},y)=\lim_{\Delta x\to\pm 0}u(x_{i+\frac{1}{2}}+\Delta x,y), \tag{A.3}$$

similarly for $u(x,y^{\pm}_{j+\frac{1}{2}})$.

We consider to use nodal basis to represent functions in the local space $\mathcal{Z}_h^{\mathbf{K}}$, and approximate the integrals by numerical quadratures. We denote $P^K(I_i)$ as $P^K(I_i):=Z_h^K|_{I_i}$. We use Lagrange basis $\{\phi_i^k(x)\}_{k=0}^K$ associated with the $K+1$ Gaussian quadrature points $\{x_i^k\}_{k=0}^K$ on I_i, defined as below

$$\phi_i^k(x)\in P^K(I_i),\quad \text{and}\quad \phi_i^k(x_i^{k'})=\delta_{kk'},\quad k,k'=0,1,\cdots,K. \tag{A.4}$$

Here $\delta_{kk'}$ represents the Kronecker delta function. We mention that in two dimensional case, we choose a tensor product of the one dimensional nodal basis, which associated with K_1+1 and K_2+1 Gaussian quadrature points on I_i and I_j along x and y directions repectively. Here we denote the corresponding quadrature weights $\{\omega_k\}_{k=0}^K$ on the reference element $(-\frac{1}{2},\frac{1}{2})$.

A.2 IMEX-NDG method for the kinetic equation

For 2D BGK equation, we discretize in time using a first-order IMEX method. We will handle the convection term $\mathbf{v}\cdot\nabla_{\mathbf{x}}f$ using an explicit method, while treating the collision term $\nu(M_U-f)/\varepsilon$ implicitly. The scheme is written as follows

$$f^{n+1}(\mathbf{v})=\frac{\varepsilon}{\varepsilon+\nu^{n+1}\Delta t}\left(f^n(\mathbf{v})-\Delta t\mathbf{v}\cdot\nabla_{\mathbf{x}}f^n(\mathbf{v})\right)+\frac{\nu^{n+1}\Delta t}{\varepsilon+\nu^{n+1}\Delta t}M(\mathbf{v},U^{n+1}), \tag{A.5}$$

with the initial condition $f^0(\mathbf{v})=f(0,\mathbf{x},\mathbf{v})$.

The implicit Maxwellian $M(\mathbf{v},U^{n+1})$ is first computed from the macroscopic quantity U^{n+1}, where

$$\mathbf{U}^{n+1}:=(\rho^{n+1},(\rho\mathbf{u})^{n+1},E^{n+1})^{\top}=\int_{\mathbb{R}^3}m(\mathbf{v})(f^n-\Delta t\mathbf{v}\cdot\nabla_{\mathbf{x}}f^n)\,d\mathbf{v}. \tag{A.6}$$

For each $\mathbf{v}$, we define

$$\mathbf{R}(\mathbf{v}):=f(\mathbf{v})-\Delta t\mathbf{v}\cdot\nabla_{\mathbf{x}}f(\mathbf{v}), \tag{A.7}$$

we generate a solution process in vector form

$$\begin{cases}\mathbf{R}^{n+1}(\mathbf{v}):=f(\mathbf{v})-\Delta t\mathbf{v}\cdot\nabla_{\mathbf{x}}f^n(\mathbf{v}),\\ \mathbf{U}^{n+1}=\displaystyle\int_{\mathbb{R}^3}m(\mathbf{v})\mathbf{R}^{n+1}(\mathbf{v})\,d\mathbf{v},\\ f^{n+1}(\mathbf{v})=\dfrac{\varepsilon}{\varepsilon+\nu^{n+1}\Delta t}R^{n+1}(\mathbf{v})+\dfrac{\nu^{n+1}\Delta t}{\varepsilon+\nu^{n+1}\Delta t}M(\mathbf{v},\mathbf{U}^{n+1}).\end{cases} \tag{A.8}$$

Following this strategy and for simplicity keeping $\mathbf{v}\in\mathbb{R}^3$, the discontinuous galerkin scheme for (A.8) can be written as: for given $f_h^n(\mathbf{v})$, we find $f_h^{n+1}(\mathbf{v})$, by computing a

discrete approximation R_h^{n+1}, which solves the following problem: for any $\xi \in \mathcal{Z}_h^{\mathbf{K}}$ and for $1 \le i \le N_x, 1 \le j \le N_y$,

$$\begin{aligned}\mathbf{R}^{n+1}_{i,k_1,j,k_2}(\mathbf{v}) = & f^n_{i,k_1,j,k_2}(\mathbf{v}) + \frac{1}{\omega_{k_1}\omega_{k_2}} \frac{\Delta t}{\Delta x_i \Delta y_i} \Bigg(\sum_{l_1=0}^{K_1} \sum_{l_2=0}^{K_2} \mathbf{v} \cdot \nabla_{\mathbf{x}} (\phi_i^{k_1}(x_i^{l_1}) \phi_j^{k_2}(y_j^{l_2})) f^n_{i,k_1,j,k_2}(\mathbf{v}) \\ & - \Delta y_j \sum_{l_2=0}^{K_2} \omega_{l_2} \Big(\widetilde{(v_1 f)}(x_{i+\frac{1}{2}}, y_j^{l_2}) \xi(x^-_{i+\frac{1}{2}}, y_j^{l_2}) - \widetilde{(v_1 f)}(x_{i-\frac{1}{2}}, y_j^{l_2}) \xi(x^+_{i-\frac{1}{2}}, y_j^{l_2}) \Big) \\ & - \Delta x_i \sum_{l_1=0}^{K_1} \omega_{l_1} \Big(\widetilde{(v_2 f)}(x_i^{l_1}, y_{i+\frac{1}{2}}) \xi(x_i^{l_1}, y^-_{i+\frac{1}{2}}) - \widetilde{(v_2 f)}(x_i^{l_1}, y_{i-\frac{1}{2}}) \xi(x_i^{l_1}, y^+_{i-\frac{1}{2}}) \Big) \Bigg),\end{aligned} \tag{A.9}$$

where Lagrange basis and corresponding weights are used instead of integral expressions. $\widetilde{vf}$ is defined to be an upwind numerical flux along its direction,

$$\widetilde{vf} := \begin{cases} vf^-, & \text{if} \quad v \ge 0, \\ vf^+, & \text{if} \quad ; v < 0, \end{cases} \tag{A.10}$$

where $f^{\pm}$ are the left and right limits of f_h^n at the cell interface of two adjacent cells respectively. $\mathbf{U}$ is given by

$$\mathbf{U}^{n+1}_{i,k_1,j,k_2} = \int_{\mathbb{R}^3} m(\mathbf{v}) \mathbf{R}^{n+1}_{i,k_1,j,k_2}(\mathbf{v})\, d\mathbf{v} \tag{A.11}$$

and the discrete approximation f_h^{n+1} is defined by

$$f^{n+1}_{i,k_1,j,k_2}(\mathbf{v}) = \frac{\varepsilon}{\varepsilon + \nu^{n+1}_{i,k_1,j,k_2}\Delta t} \mathbf{R}^{n+1}_{i,k_1,j,k_2}(\mathbf{v}) + \frac{\nu^{n+1}_{i,k_1,j,k_2}\Delta t}{\varepsilon + \nu^{n+1}_{i,k_1,j,k_2}\Delta t} M(\mathbf{v}, \mathbf{U}^{n+1}_{i,k_1,j,k_2}), \tag{A.12}$$

where $f^n_{i,k_1,j,k_2}(\mathbf{v}), \mathbf{R}^{n+1}_{i,k_1,j,k_2}(\mathbf{v}), \mathbf{U}^{n+1}_{i,k_1,j,k_2}$ with subindex (i,k_1,j,k_2) are the corresponding numerical flux at the $(k_1,k_2)-$th Gaussian quadrature point in cell $I_{i,j}$, for $0 \le k_1 \le K_1$ and $0 \le k_2 \le K_2$.

A.3 NDG method for the compressible Navier-stokes equations

We will produce a discontinuous Galerkin scheme for the compressible Navier–Stokes equation Eq. (A.13)

$$\begin{cases} \partial_t \rho + \nabla_{\mathbf{x}} \cdot (\rho \mathbf{u}) = 0, \\ \partial_t (\rho \mathbf{u}) + \nabla_{\mathbf{x}} \cdot (\rho \mathbf{u} \otimes \mathbf{u} + \rho T \mathbf{I}) = \varepsilon \nabla_{\mathbf{x}} \cdot (\sigma(\mathbf{u})), \\ \partial_t E + \nabla_{\mathbf{x}} \cdot (\mathbf{u}(E + \rho T)) = \varepsilon \nabla_{\mathbf{x}} \cdot (\sigma(\mathbf{u}) \cdot \mathbf{u} + \mathbf{q}), \end{cases} \tag{A.13}$$

where $\sigma := -\mu \mathbf{D}(\mathbf{u})$ is the viscosity tensor and $\mathbf{q} := -\kappa \nabla_{\mathbf{x}} T$ is the heat flux.

Letting $\nabla_{\mathbf{x}} \mathbf{U} := (\mathbf{S}_1, \mathbf{S}_2)$, with a forward Euler time discretization, the NDG scheme for (A.13) is defined as follows: find $\mathbf{U}_h^n \in \mathbf{Z}_h^{\mathbf{K}}$, such that for any $\eta(\mathbf{x}) \in \mathcal{Z}_h^{\mathbf{K}}$ and $1 \le i \le N_x, 1 \le j \le N_y$, we have

$$\int_{I_{i,j}} \mathbf{U}_h^{n+1} \eta(\mathbf{x})\, d\mathbf{x} = \int_{I_{i,j}} \mathbf{U}_h^n \eta(\mathbf{x})\, d\mathbf{x} + \Delta t \left(\int_{I_{i,j}} \mathbf{F}(\mathbf{U}_h^n, \mathbf{S}_h^n) \cdot \nabla_{\mathbf{x}} \eta(\mathbf{x})\, d\mathbf{x} - \oint_{\partial I_{i,j}} \eta(\mathbf{x}) \widehat{\mathbf{F}} \cdot \mathbf{n}\, d\sigma \right), \quad \text{(A.14)}$$

where $\nabla_{\mathbf{x}} \mathbf{U}_h^n = (\mathbf{S}_{1,h}^n, \mathbf{S}_{2,h}^n)$ is such that $\mathbf{S}_{1,h}^n, \mathbf{S}_{2,h}^n \in \mathbf{Z}_h^{\mathbf{K}}$ for any $\eta(\mathbf{x}) \in \mathcal{Z}_h^{\mathbf{K}}$ and $1 \le i \le N_x, 1 \le j \le N_y$, we have

$$\int_{I_{i,j}} \mathbf{S}_{1,h}^n \eta(\mathbf{x})\, d\mathbf{x} = -\int_{I_{i,j}} \mathbf{U}_h^n \partial_x \eta(\mathbf{x})\, d\mathbf{x} + \int_{I_j} \eta(x_{i+\frac{1}{2}}^-, y) \widehat{\mathbf{U}}(x_{i+\frac{1}{2}}, y) - \eta(x_{i-\frac{1}{2}}^+, y) \widehat{\mathbf{U}}(x_{i-\frac{1}{2}}, y)\, dy, \quad \text{(A.15a)}$$

$$\int_{I_{i,j}} \mathbf{S}_{2,h}^n \eta(\mathbf{x})\, d\mathbf{x} = -\int_{I_{i,j}} \mathbf{U}_h^n \partial_y \eta(\mathbf{x})\, d\mathbf{x} + \int_{I_i} \eta(x, y_{i+\frac{1}{2}}^-) \widehat{\mathbf{U}}(x, y_{i+\frac{1}{2}}) - \eta(x, y_{i-\frac{1}{2}}^+) \widehat{\mathbf{U}}(x, y_{i-\frac{1}{2}})\, dx, \quad \text{(A.15b)}$$

where $\widehat{\mathbf{U}}$ is taken to be a central flux

$$\widehat{\mathbf{U}} := \frac{1}{2}(\mathbf{U}^+ + \mathbf{U}^-),$$

along x or y direction. $\mathbf{U}^\pm$ are the left and right limits of $\mathbf{U}_h^n$ at the cell interface. Finally we apply a quadrature formula in the previous section in order to get the nodal discontinuous galerkin scheme.

Remark A.1. The interface coupling condition, which can be found detailed in [13], is omitted here. Moreover, we follow [12] to establish the hydrodynamic solvers for 1D BGK equation.

References

[1] I. D. Boyd, G. Chen, G. V. Candler, Predicting failure of the continuum fluid equations in transitional hypersonic flows, *Physics of fluids* 7 (1) (1995) 210–219.

[2] V. Kolobov, R. Arslanbekov, V. V. Aristov, A. Frolova, S. A. Zabelok, Unified solver for rarefied and continuum flows with adaptive mesh and algorithm refinement, *Journal of Computational Physics* 223 (2) (2007) 589–608.

[3] P. Degond, G. Dimarco, Fluid simulations with localized boltzmann upscaling by direct simulation monte-carlo, *Journal of Computational Physics* 231 (6) (2012) 2414–2437.

[4] S. Tiwari, Coupling of the boltzmann and euler equations with automatic domain decomposition, *Journal of Computational Physics* 144 (2) (1998) 710–726.

[5] P. Degond, G. Dimarco, L. Mieussens, A multiscale kinetic–fluid solver with dynamic localization of kinetic effects, *Journal of Computational Physics* 229 (13) (2010) 4907–4933.

[6] S. Tiwari, A. Klar, S. Hardt, A particle–particle hybrid method for kinetic and continuum equations, *Journal of Computational Physics* 228 (18) (2009) 7109–7124.

[7] S. Tiwari, A. Klar, S. Hardt, Simulations of micro channel gas flows with domain decomposition technique for kinetic and fluid dynamics equations, in: Domain Decomposition Methods in Science and Engineering XXI, Springer, 2014, pp. 227–236.
[8] A. Alaia, G. Puppo, A hybrid method for hydrodynamic-kinetic flow–Part II–Coupling of hydrodynamic and kinetic models, *Journal of Computational Physics* 231 (16) (2012) 5217–5242.
[9] S. Chen, E. Weinan, Y. Liu, C.-W. Shu, A discontinuous galerkin implementation of a domain decomposition method for kinetic-hydrodynamic coupling multiscale problems in gas dynamics and device simulations, *Journal of Computational Physics* 225 (2) (2007) 1314–1330.
[10] F. Filbet, T. Rey, A hierarchy of hybrid numerical methods for multiscale kinetic equations, *SIAM Journal on Scientific Computing* 37 (3) (2015) A1218–A1247.
[11] C. D. Levermore, W. J. Morokoff, B. T. Nadiga, Moment realizability and the validity of the Navier–Stokes equations for rarefied gas dynamics, *Physics of Fluids* 10 (12) (1998) 3214–3226.
[12] T. Xiong, J.-M. Qiu, A hierarchical uniformly high order DG-IMEX scheme for the 1D BGK equation, *Journal of Computational Physics* 336 (2017) 164–191.
[13] F. Filbet, T. Xiong, A hybrid discontinuous Galerkin scheme for multi-scale kinetic equations, *Journal of Computational Physics* 372 (2018) 841–863.
[14] F. Rosenblatt, The perceptron: a probabilistic model for information storage and organization in the brain., *Psychological review* 65 (6) (1958) 386.
[15] A. Khashman, A neural network model for credit risk evaluation, *International Journal of Neural Systems* 19 (04) (2009) 285–294.
[16] A. U. S. Khan, N. Akhtar, M. N. Qureshi, Real-time credit-card fraud detection using artificial neural network tuned by simulated annealing algorithm, in: Proceedings of international conference on recent trends in information, telecommunication and computing, ITC, Citeseer, 2014, pp. 113–121.
[17] D. H. Mantzaris, G. C. Anastassopoulos, D. K. Lymberopoulos, Medical disease prediction using artificial neural networks, in: 2008 8th IEEE International Conference on BioInformatics and BioEngineering, IEEE, 2008, pp. 1–6.
[18] S. Punitha, F. Al-Turjman, T. Stephan, An automated breast cancer diagnosis using feature selection and parameter optimization in ANN, *Computers & Electrical Engineering* 90 (2021) 106958.
[19] D. S. Boone, M. Roehm, Retail segmentation using artificial neural networks, *International journal of research in marketing* 19 (3) (2002) 287–301.
[20] A. Krizhevsky, I. Sutskever, G. E. Hinton, Imagenet classification with deep convolutional neural networks, *Communications of the ACM* 60 (6) (2017) 84–90.
[21] T. Sercu, C. Puhrsch, B. Kingsbury, Y. LeCun, Very deep multilingual convolutional neural networks for LVCSR, in: 2016 IEEE international conference on acoustics, speech and signal processing (ICASSP), IEEE, 2016, pp. 4955–4959.
[22] R. Girshick, J. Donahue, T. Darrell, J. Malik, Rich feature hierarchies for accurate object detection and semantic segmentation, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2014, pp. 580–587.
[23] S. Ren, K. He, R. Girshick, J. Sun, Faster R-CNN: Towards real-time object detection with region proposal networks, *IEEE transactions on pattern analysis and machine intelligence* 39 (6) (2016) 1137–1149.
[24] Y. Taigman, M. Yang, M. Ranzato, L. Wolf, Deepface: Closing the gap to human-level performance in face verification, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2014, pp. 1701–1708.
[25] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural computation* 9 (8) (1997)

1735–1780.
[26] F. A. Gers, J. Schmidhuber, F. Cummins, Learning to forget: Continual prediction with LSTM, *Neural computation* 12 (10) (2000) 2451–2471.
[27] G. Cybenko, Approximation by superpositions of a sigmoidal function, *Mathematics of control, signals and systems* 2 (4) (1989) 303–314.
[28] D. E. Rumelhart, G. E. Hinton, R. J. Williams, Learning representations by back-propagating errors, *nature* 323 (6088) (1986) 533–536.
[29] J. Yu, J. S. Hesthaven, A data-driven shock capturing approach for discontinuous Galekin methods, *Computers & Fluids* 245 (2022) 105592.
[30] T. Xiao, S. Schotthöfer, M. Frank, Predicting continuum breakdown with deep neural networks, *Journal of Computational Physics* 489 (2023) 112278.
[31] P. L. Bhatnagar, E. P. Gross, M. Krook, A model for collision processes in gases. I. Small amplitude processes in charged and neutral one-component systems, *Physical review* 94 (3) (1954) 511.
[32] K. Aoki, N. Masukawa, Gas flows caused by evaporation and condensation on two parallel condensed phases and the negative temperature gradient: Numerical analysis by using a nonlinear kinetic equation, *Physics of Fluids* 6 (3) (1994) 1379–1395.
[33] M. Shanker, M. Y. Hu, M. S. Hung, Effect of data standardization on neural network training, *Omega* 24 (4) (1996) 385–397.